

Tong Xiao

Jingbo Zhu

# Natural Language Processing

Neural Networks and Large Language Models

NATURAL LANGUAGE PROCESSING LAB

NORTHEASTERN UNIVERSITY

&

NIUTRANS RESEARCH

*October 5, 2026*

Copyright © 2021-2026 Tong Xiao and Jingbo Zhu

NATURAL LANGUAGE PROCESSING LAB, NORTHEASTERN UNIVERSITY  
&  
NIUTRANS RESEARCH

<https://github.com/NiuTrans/NLPBook>

<https://niutrans.github.io/NLPBook>

Licensed under the Creative Commons Attribution-NonCommercial 4.0 Unported License (the “License”). You may not use this file except in compliance with the License. You may obtain a copy of the License at <http://creativecommons.org/licenses/by-nc/4.0>. Unless required by applicable law or agreed to in writing, software distributed under the License is distributed on an “AS IS” BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied. See the License for the specific language governing permissions and limitations under the License.

*October 5, 2026*

# Chapter 1

## Foundations of Machine Learning

*The goal of machine learning is to develop methods that can automatically detect patterns in data, and then to use the uncovered patterns to predict future data or other outcomes of interest.*

– [Murphy \[2012\]](#)

*Machine learning can be broadly defined as computational methods using experience to improve performance or to make accurate predictions.*

– [Mohri et al. \[2018\]](#)

Data-driven NLP fits the above definitions<sup>1</sup>. It teaches computers to learn language *experience* from corpora, and to understand and utilize language based on that *experience*. Connecting machine learning (ML) with natural language processing is much more than a means that makes computers mimic human language intelligence from data. It is leading a revolution in both areas: natural language processing evolves by using a powerful tool of deriving meaning from corpora, and machine learning evolves by addressing the NLP challenges and testing on real-world data.

In this chapter, we present several basic concepts and models in machine learning. There are no tough bits but some preliminaries for the subsequent chapters. Here we focus on how to apply machine learning to NLP problems, in particular how to define an NLP problem as a statistical learning problem. To do this, we start with classification — one of the most widely-used examples in most introductory books. We then present several fundamental issues of machine learning. They are followed by a discussion on NLP problems from the machine learning perspective.

---

<sup>1</sup>We drop the term *data-driven* from now on and assume that all NLP models are data-driven in the remainder of this book.

## 1.1 Math Basics

In the remainder of this chapter and the following chapters, we will talk about machine learning problems using the tool of applied mathematics. Here are the math basics. If you find the details trivial, you can skip to Section 1.2 directly.

### 1.1.1 Linear Algebra

#### 1. Vectors and Matrices

A **scalar** may or may not be the simplest concept in linear algebra, but is surely the most common concept that one learns in high school or in university. A scalar is a number. It is a quantity that has a magnitude but has no direction. For example, height, weight, distance, temperature are all examples of scalars. Here we use an italic number to denote a scalar, for example,  $a$ ,  $b$ ,  $x$ ,  $A$ , and so on.

**Vector** and **matrix** are defined in terms of scalar. A vector is an array of scalars, or simply a number list. A matrix is a rectangular array of scalars. In this book, we follow the convention of using bold letters to denote vectors and matrices. For example, an  $n$ -dimensional vector can be written as

$$\mathbf{a} = \begin{bmatrix} a_1 & a_2 & \cdots & a_n \end{bmatrix}, \quad (1.1)$$

where  $\{a_1, a_2, \dots, a_n\}$  are the elements (or entries) of the vector. Each element corresponds to a dimension. For convenience of notation, we write  $a_i$  as  $a(i)$  sometimes. A vector is a real-valued vector only if all the elements are real numbers (i.e.,  $a_i \in \mathbb{R}$  for each  $i$ ), denoted as  $\mathbf{a} \in \mathbb{R}^n$ . Likewise, we can write an  $m \times n$  matrix as

$$\mathbf{A} = \begin{bmatrix} A_{11} & A_{12} & \cdots & A_{1n} \\ A_{21} & A_{22} & \cdots & A_{2n} \\ \cdots & \cdots & \cdots & \cdots \\ A_{m1} & A_{m2} & \cdots & A_{mn} \end{bmatrix}, \quad (1.2)$$

where  $m$  is the number of rows and  $n$  is the number of columns.  $A_{ij}$  is the entry  $(i, j)$  of the matrix. A real-valued matrix is denoted as  $\mathbf{A} \in \mathbb{R}^{m \times n}$ . Occasionally, we use  $\mathbf{A}_{m \times n}$  to emphasize that the shape of the matrix is  $m \times n$ .

There are a few special matrices. For example, a matrix whose elements are all zeros is a **zero matrix**, denoted as  $\mathbf{0}$ . Another example is the **identity matrix**, denoted as  $\mathbf{I}$ . It is a square matrix whose diagonal elements are all 1, and other elements are 0. Vectors can be treated as a special sort of matrices, too. For example, the vector in Eq. (1.1) is a matrix with only one row.

## 2. Matrix Transpose

The **transpose** of a matrix  $A_{m \times n}$  is a matrix  $B_{n \times m}$  subject to  $A_{ij} = B_{ji}$  for each pair of  $i$  and  $j$ . Often,  $\mathbf{A}$ 's transpose is denoted as  $\mathbf{A}^\top$ . For example, for a matrix

$$\mathbf{A} = \begin{bmatrix} 8 & 0 & 0 \\ 2 & 9 & 7 \end{bmatrix}, \quad (1.3)$$

the transpose is

$$\mathbf{A}^\top = \begin{bmatrix} 8 & 2 \\ 0 & 9 \\ 0 & 7 \end{bmatrix}. \quad (1.4)$$

One can transpose a vector as well. For a vector

$$\mathbf{a} = \begin{bmatrix} 1 & 9 & 7 & 3 \end{bmatrix} \quad (1.5)$$

the transpose is

$$\mathbf{a}^\top = \begin{bmatrix} 1 \\ 9 \\ 7 \\ 3 \end{bmatrix} \quad (1.6)$$

In general, a vector with only one row is called a **row vector** (as in Eq. (1.5)), and a vector with only one column is called a **column vector** (as in Eq. (1.6)). In this book, all vectors are row vectors by default.

## 3. Element-wise Operations on Matrices

Suppose  $\mathbf{A}$  and  $\mathbf{B}$  are two matrices of the same shape, say  $\mathbf{A}, \mathbf{B} \in \mathbb{R}^{m \times n}$ . The **matrix addition** of  $\mathbf{A}$  and  $\mathbf{B}$  is written as  $\mathbf{A} + \mathbf{B}$ .  $\mathbf{A} + \mathbf{B}$  is a matrix in  $\mathbb{R}^{m \times n}$  such that each element is the sum of the corresponding elements of  $\mathbf{A}$  and  $\mathbf{B}$ . Here is an example.

$$\begin{aligned} \mathbf{A} + \mathbf{B} &= \begin{bmatrix} 8 & 0 & 0 \\ 2 & 9 & 7 \end{bmatrix} + \begin{bmatrix} 1 & 1 & 1 \\ 1 & 0 & 4 \end{bmatrix} \\ &= \begin{bmatrix} 9 & 1 & 1 \\ 3 & 9 & 11 \end{bmatrix}. \end{aligned} \quad (1.7)$$

In a similar way, we can define element-wise subtraction ( $\mathbf{A} - \mathbf{B}$ ), product ( $\mathbf{A} \odot \mathbf{B}$ ), division ( $\mathbf{A} \oslash \mathbf{B}$ ) and other operations. A special case of element-wise operations is **scalar multiplication**, where we multiply a matrix  $\mathbf{A}$  with a scalar  $k$ , denoted as  $k \times \mathbf{A}$  or  $k\mathbf{A}$ . See

below for an example for  $k = 2$  and  $\mathbf{A} = \begin{bmatrix} 8 & 0 & 0 \\ 2 & 9 & 7 \end{bmatrix}$ .

$$\begin{aligned} k\mathbf{A} &= 2 \begin{bmatrix} 8 & 0 & 0 \\ 2 & 9 & 7 \end{bmatrix} \\ &= \begin{bmatrix} 16 & 0 & 0 \\ 4 & 18 & 14 \end{bmatrix}. \end{aligned} \quad (1.8)$$

Let  $\mathbf{A}$ ,  $\mathbf{B}$  and  $\mathbf{C}$  be matrices in  $\mathbb{R}^{m \times n}$ , and  $k$  and  $l$  be scalars in  $\mathbb{R}$ . Some properties of the matrix operations are:

- Property of the zero matrix:

$$\begin{aligned} \mathbf{A} &= \mathbf{A} + \mathbf{0} \\ &= \mathbf{0} + \mathbf{A}. \end{aligned} \quad (1.9)$$

- Commutativity:

$$\mathbf{A} + \mathbf{B} = \mathbf{B} + \mathbf{A} \quad (1.10)$$

$$kl\mathbf{A} = lk\mathbf{A}. \quad (1.11)$$

- Associativity:

$$(\mathbf{A} + \mathbf{B}) + \mathbf{C} = \mathbf{A} + (\mathbf{B} + \mathbf{C}) \quad (1.12)$$

$$(kl)\mathbf{A} = l(k\mathbf{A}). \quad (1.13)$$

- Distributivity:

$$k(\mathbf{A} + \mathbf{B}) = k\mathbf{A} + k\mathbf{B} \quad (1.14)$$

$$(k + l)\mathbf{A} = k\mathbf{A} + l\mathbf{A}. \quad (1.15)$$

#### 4. Dot Product

The **dot product** of two vectors of the same dimensions  $\mathbf{a} = \begin{bmatrix} a_1 & a_2 & \cdots & a_n \end{bmatrix}$  and  $\mathbf{b} = \begin{bmatrix} b_1 & b_2 & \cdots & b_n \end{bmatrix}$  is defined to be:

$$\begin{aligned} \mathbf{a} \cdot \mathbf{b} &= \sum_{i=1}^n a_i b_i \\ &= a_1 b_1 + a_2 b_2 + \cdots + a_n b_n. \end{aligned} \quad (1.16)$$

In geometry, a real-valued vector  $\mathbf{a}$  can be seen as a geometric object having both magnitude (denoted as  $|\mathbf{a}|$ ) and direction. The dot product of  $\mathbf{a}$  and  $\mathbf{b}$  can also be defined as:

$$\mathbf{a} \cdot \mathbf{b} = |\mathbf{a}| \times |\mathbf{b}| \times \cos(\theta), \quad (1.17)$$

where  $\theta$  is the angle between  $\mathbf{a}$  and  $\mathbf{b}$ .

## 5. Matrix Product

The **matrix product** (or **matrix-matrix product**) operates on two matrices. Given a matrix  $\mathbf{A} \in \mathbb{R}^{m \times p}$  and a matrix  $\mathbf{B} \in \mathbb{R}^{p \times n}$ , the matrix product of  $\mathbf{A}$  and  $\mathbf{B}$  produces a matrix  $\mathbf{C} \in \mathbb{R}^{m \times n}$  whose elements are defined as:

$$\begin{aligned} C_{ij} &= \sum_{k=1}^p A_{ik} \times B_{kj} \\ &= A_{i1} \times B_{1j} + A_{i2} \times B_{2j} + \cdots + A_{ip} \times B_{pj}. \end{aligned} \quad (1.18)$$

Matrix product requires that the number of columns in  $\mathbf{A}$  is exactly the same as the number of rows in  $\mathbf{B}$ . In this book we use  $\mathbf{AB}$  to denote the matrix product of  $\mathbf{A}$  and  $\mathbf{B}$ . Here are a few properties of matrix product.

- **Distributivity.** For  $\mathbf{A} \in \mathbb{R}^{m \times p}$  and  $\mathbf{B}, \mathbf{C} \in \mathbb{R}^{p \times n}$ , the left distributivity is defined as:

$$\mathbf{A}(\mathbf{B} + \mathbf{C}) = \mathbf{AB} + \mathbf{AC}. \quad (1.19)$$

For  $\mathbf{A}, \mathbf{B} \in \mathbb{R}^{m \times p}$  and  $\mathbf{C} \in \mathbb{R}^{p \times n}$ , the right distributivity is defined as:

$$(\mathbf{A} + \mathbf{B})\mathbf{C} = \mathbf{AC} + \mathbf{BC}. \quad (1.20)$$

- **Associativity.** For  $\mathbf{A} \in \mathbb{R}^{m \times p}$ ,  $\mathbf{B} \in \mathbb{R}^{p \times q}$  and  $\mathbf{C} \in \mathbb{R}^{q \times n}$ , the associativity defines

$$(\mathbf{AB})\mathbf{C} = \mathbf{A}(\mathbf{BC}). \quad (1.21)$$

- **Transpose.** For  $\mathbf{A} \in \mathbb{R}^{m \times p}$  and  $\mathbf{B} \in \mathbb{R}^{p \times n}$ , we have

$$(\mathbf{AB})^\top = \mathbf{B}^\top \mathbf{A}^\top. \quad (1.22)$$

Matrix product is not commutative, i.e., we do not have  $\mathbf{AB} = \mathbf{BA}$  for all  $\mathbf{A}$  and  $\mathbf{B}$  even if  $\mathbf{A}$  and  $\mathbf{B}$  are square matrices with the same shape. Based on matrix-matrix product, we can define vector-matrix product and matrix-vector product accordingly. This is trivial because all we need is to see a vector as a matrix in multiplication.

## 6. Norm

In a vector space, a **norm** is a function used to measure the “length” of a vector. For a vector  $\mathbf{a} \in \mathbb{R}^n$ , the  $p$ -norm (or  $l_p$  norm) is defined as:

$$\|\mathbf{a}\|_p = \left( \sum_{i=1}^n |a_i|^p \right)^{1/p}. \quad (1.23)$$

The most commonly used  $p$ -norms are for  $p = 1, 2$ , and  $\infty$ :

$$\|\mathbf{a}\|_1 = \sum_{i=1}^n |a_i| \quad (1.24)$$

$$\|\mathbf{a}\|_2 = \sqrt{\sum_{i=1}^n |a_i|^2} \quad (1.25)$$

$$\|\mathbf{a}\|_\infty = \max\{|a_1|, |a_2|, \dots, |a_n|\}. \quad (1.26)$$

Norms can also measure the distance between two points. Let  $\mathbf{b} \in \mathbb{R}^n$  be another vector. The  $p$ -norm distance between  $\mathbf{a}$  and  $\mathbf{b}$  is  $\|\mathbf{a} - \mathbf{b}\|_p$ .

## 1.1.2 Probability and Statistics

### 1. What is Probability

**Probability** is a measure of uncertainty. It is a quantity that describes how likely an event is to happen. For example, if the event is certain to happen, we will say that the probability is 1; if the event will never happen, we will say that the probability is 0.

An event is the outcome of a random experiment. For example, in a coin-tossing experiment, an event could be "the coin lands heads". A set of related events is described by a **random variable** or **variable** for short. For example, we can define the outcome of tossing a coin as a variable  $x$ . As there are two outcomes (heads or tails), we have two choices for the value of  $x$ . We can define that  $x = 1$  when the coin lands heads, and  $x = 0$  otherwise. Hence,  $x$  is a **binary variable** or more precisely a 0-1 variable. A random variable taking a value means that an event happens. For example,  $x = 1$  means the event of the coin landing heads.

In mathematics, probability is defined as a **probability measure** over a sample space. This measure must satisfy certain axioms, such as countable additivity. While we skip the measure-theoretic details here, we treat probability as a function that maps events to real numbers in the range  $[0, 1]$ .

Usually, a probability measure is denoted as a function  $\Pr(\cdot)$ , called a **probability function**. When the input of  $\Pr(\cdot)$  is defined on a discrete set of events, the output of the function is the probability that an event happens. For example,  $\Pr(x = 1)$  means the probability of  $x$  equalling 1. Note that  $\Pr(x)$  is a function that varies its output by choosing different values of  $x$ . Suppose  $x_1$  is a value that  $x$  can take. When we write  $\Pr(x_1)$ , it means  $\Pr(x = x_1)$ . Because the probability over all events should be 1, any probability function should be subject to:

$$\sum_{x_i \in X} \Pr(x_i) = 1, \quad (1.27)$$

where  $X$  is the set of all events. A probability function can be defined on two variables or more. Here are a few widely-used cases.

- **Joint Probability.** It is the probability that two events  $x$  and  $y$  happen at the same time,

denoted as  $\Pr(x, y)$ . If  $x$  and  $y$  are independent, their joint probability is the product of their individual probabilities

$$\Pr(x, y) = \Pr(x) \Pr(y). \quad (1.28)$$

- **Conditional Probability.** It is the probability that  $x$  happens in the presence of  $y$  happening, denoted as  $\Pr(x|y)$ .  $\Pr(x|y)$  can be defined as:

$$\Pr(x|y) = \frac{\Pr(x, y)}{\Pr(y)}. \quad (1.29)$$

- **Marginal Probability.** It is another way to define the probability of a single variable. Given the joint probability on two variables, the marginal probability is defined as

$$\Pr(x) = \sum_{y_j \in Y} \Pr(x, y_j), \quad (1.30)$$

where  $Y$  is the event space of  $y_j$ . Eq. (1.30) says that  $\Pr(x)$  is marginalized over  $Y$ .

Another note on joint probability. In some cases, one would like to use conditional probabilities to represent a joint probability. To this end, one can rewrite the joint probability by the **product rule** or the **chain rule**, like this

$$\Pr(x, y) = \Pr(x|y) \Pr(y). \quad (1.31)$$

For continuous variables, the probability of the variable taking any exact point value is zero. Instead, we have a density for that point. More formally, given a continuous variable  $x$ , a **probability density** of  $x$  is written as  $\Pr(x)$ . Suppose  $x \in \mathbb{R}$ . The probability of  $x$  lying in the interval  $[a, b]$  is defined via an integral:

$$\Pr(x \in [a, b]) = \int_a^b \Pr(x) dx. \quad (1.32)$$

Obviously, we have

$$\int_{-\infty}^{+\infty} \Pr(x) dx = 1. \quad (1.33)$$

For other properties, such as joint probability and conditional probability, the forms for continuous variables are almost the same as those for discrete variables. We just need to replace the sums in the formulas with the integrals.

## 2. Distribution and Expectation

A **probability distribution** (or simply **distribution**) describes how probabilities are assigned to the possible values of a random variable. It is defined by a **probability mass function (PMF)** for discrete variables or a **probability density function (PDF)** for continuous variables.

For example, a uniform distribution on a discrete variable  $x$  that takes values from  $\{x_1, x_2\}$  can be described as  $\Pr(x_i) = 1/2$  because  $\Pr(x_1) = \Pr(x_2)$  and  $\Pr(x_1) + \Pr(x_2) = 1$ ; A uniform distribution on a continuous variable in  $[-2, 2]$  can be described as  $\Pr(x) = 1/4$  because probability density function  $\Pr(x)$  is a constant for any  $x \in [-2, 2]$  and  $\int_{-2}^2 \Pr(x)dx = 1$ . Statisticians have developed many distributions for describing the world we are living in, such as binomial distribution, Bernoulli distribution and Gaussian/normal distribution. One can find details of these distributions in most textbooks on statistics [McClave and Sincich, 2006; Freedman et al., 2007].

For describing properties of a variable, a popular means is to compute the **expected value** or **expectation** of the variable. Let  $x$  be a discrete variable that takes values from  $\{x_1, \dots, x_n\}$ , and  $\Pr(x)$  be a distribution on  $x$ . The expected value of  $x$  is defined to be:

$$\mathbb{E}_{x \sim \Pr(x)}(x) = \sum_{i=1}^n x_i \cdot \Pr(x_i), \quad (1.34)$$

where the subscript  $x \sim \Pr(x)$  indicates that  $x$  follows the distribution  $\Pr(x)$ . In many cases, we can drop the subscript and rewrite it as  $\mathbb{E}(x)$ .  $\mathbb{E}(x)$  is essentially the weighted average value of  $x$  under the distribution  $\Pr(x)$ . It is a measure of central tendency, and is sometimes called the **mean** of a variable (denoted as  $\mu$ ).

Then, we can define the **variance** of a variable as the expected squared deviation from its mean

$$\text{Var}(x) = \mathbb{E}[(x - \mathbb{E}(x))^2]. \quad (1.35)$$

Informally, it describes how far the values are from the mean.  $\text{Var}(x)$  is usually written as  $\sigma^2$ , where  $\sigma$  is called **standard deviation**.

For a continuous variable  $x \in \mathbb{R}$ , the expected value is defined as:

$$\mathbb{E}(x) = \int_{-\infty}^{+\infty} x \cdot \Pr(x)dx, \quad (1.36)$$

where  $\Pr(x)$  is a probability density function. For computing the variance of  $x$ , we just reuse Eq. (1.35).

### 3. Entropy

Entropy is one of the most important tools of describing random variables and processes [Shannon, 1948]. It is a measure of **expected surprisal**. The more deterministic the outcomes are, the less surprise and less information there will be. For simplicity, we restrict the discussion to discrete variables here<sup>2</sup>. Let  $x$  be a variable and  $\Pr(x)$  be a distribution on  $x$ . The entropy is

---

<sup>2</sup>For continuous variables, we have similar calculations.

written as:

$$H(x) = -\sum_{i=1}^n \Pr(x_i) \cdot \log_b \Pr(x_i), \quad (1.37)$$

where  $b$  is the base of the logarithm function. The value of  $b$  is typically set to 2, 10 and  $e$ .

Beyond measuring the uncertainty of a single distribution, we can quantify the dissimilarity between two distributions. Suppose  $p(x)$  and  $q(x)$  are distributions on  $x$ . The **relative entropy** of  $p$  with respect to  $q$  is defined to be:

$$D_{\text{KL}}(p \| q) = \sum_{i=1}^n p(x_i) \cdot \log_b \frac{p(x_i)}{q(x_i)}. \quad (1.38)$$

We can treat  $p(x_i)$  as a weight to the log likelihood ratio  $\log_b \frac{p(x_i)}{q(x_i)}$ . Hence,  $D_{\text{KL}}(p \| q)$  is a weighted sum of the likelihood ratios over all possible values. A smaller value  $D_{\text{KL}}(p \| q)$  indicates that distributions  $p$  and  $q$  are closer. For example,  $p$  and  $q$  will be identical if  $D_{\text{KL}}(p \| q) = 0$ . The relative entropy is also called the **Kullback-Leibler (KL) divergence**. Note that the relative entropy is asymmetric, i.e., we cannot guarantee  $D_{\text{KL}}(p \| q) = D_{\text{KL}}(q \| p)$ .

Another concept that is popular in machine learning is **cross-entropy**. It is a measure of the information (in terms of the total number of bits) that we need to transmit the events from a source in one distribution with another distribution. More formally, we write the cross-entropy of the distribution  $p$  with the distribution  $q$  as  $H_{\text{cross}}(p, q)$ . It is defined to be:

$$H_{\text{cross}}(p, q) = -\sum_{i=1}^n p(x_i) \cdot \log_2 q(x_i). \quad (1.39)$$

Like relative entropy, cross-entropy is asymmetric. In the context of optimization, such as training NLP systems, both are frequently used as objective functions. The relationship between them is given by  $H(p, q) = H(p) + D_{\text{KL}}(p \| q)$ . While relative entropy measures the **additional** bits required due to using  $q$  instead of  $p$ , cross-entropy measures the total bits required.

## 1.2 Designing a Text Classifier

**Classification** is one of the most common problems in machine learning. It aims at automatically categorizing objects into a set of classes. These classes are called **labels**, or **tags**, or **categories**. In general, a program that performs classification is called a **classifier** or **classification system**. There are a vast number of practical applications of classifiers. A simple example is spam filtering where one needs to label an email as “spam” or “not-spam”. More challenging examples include classifying computed tomography images of organs into “normal” or “not-normal”, determining whether a piece of Chinese text is written by native speakers or not, labeling a patent application with a set of the IPC codes it belongs to, and so on.

Many machine learning theories and algorithms are modeled and tested on classification tasks. Following this convention, we consider text classification as an example to get started. Assume that we have a corpus like this.

| Text                                                                 | Label    |
|----------------------------------------------------------------------|----------|
| The game was wonderful.                                              | Not-food |
| I've tried my best to recreate it in my kitchen. It tastes heavenly. | Food     |
| For centuries seaweed was considered a food for normal people.       | Food     |
| Have you finished your coding work today?                            | Not-Food |
| I was wondering how you could miss the bus.                          | Not-Food |
| I like fruit because it is good for health.                          | Food     |
| Natural language processing research is amazing.                     | Not-Food |
| ...                                                                  | ...      |

Each line of the corpus is a tuple consisting of a piece of text (hereafter referred to as a document) and a label that indicates whether the text is about food or not. We call such tuples **samples**, or more precisely **labeled samples**. Labeled samples are essentially question-answer pairs although they do not strictly follow the standard forms of questions and answers. For example, in the samples presented above, one can take a document as a question and take its label as the answer. In the next few chapters, we will show that such a formulation is general and fits most problems in NLP.

Next, let us assume that we have a classifier that learns how to label documents from these samples. The classifier is then used to label new, unseen documents as “Food” or “Not-Food”. For example, for the text

Fruit is not my favorite but I can enjoy it.

the classifier would categorize it as “Food”.

However, text classification, though it seems simple on the surface, is much more than merely sorting unlabeled samples into classes. It presents a wide variety of issues, especially when considering the ambiguities and richness of language. Modern classifiers are not systems comprising a set of hand-crafted rules. Instead, they model the classification problem in a probabilistic manner, making it possible to learn to classify from large-scale labeled data.

### 1.2.1 Problem Formulation

Let  $x$  be a document and  $c$  be a label. Here we assume a probabilistic classifier which would estimate how likely we choose  $c$  as the label of  $x$ , denoted as  $\Pr(c|x)$ .  $\Pr(c|x)$  is in general a **classification model**. It describes a distribution over the set of all possible labels, satisfying

$$\sum_{c \in \mathcal{C}} \Pr(c|x) = 1, \quad (1.40)$$

where  $\mathcal{C}$  is the label set. For any document, we choose the most probable label as output via

the classification model, like this

$$\hat{c} = \operatorname{argmax}_{c \in \mathcal{C}} \Pr(c|x), \quad (1.41)$$

where  $\hat{c}$  is the “best” label predicted by the model.  $\operatorname{argmax}$  is the abbreviation of the arguments of the maxima. It returns the value of the argument that maximizes some function.

Eq. (1.41) is the fundamental equation of classification. It implies three problems

- **The modeling problem.**  $\Pr(c|x)$  is a computational challenge because it is not obvious how to obtain the value of  $\Pr(c|x)$  for each pair of  $x$  and  $c$ . To make an adequate model, one may need to represent  $x$  and  $c$  in a computationally tractable way and define a mathematical relationship connecting them.
- **The learning problem.** From a statistical learning point of view, the general form of  $\Pr(c|x)$  represents a range of models configured with different variables or parameters. These models are essentially of the same form but would behave differently if we choose different values of those parameters. Thus, we need to choose a “good” model among them. This is typically addressed by optimizing the parameters on labeled data by some criterion.
- **The prediction problem.** We are addressing a **binary classification** problem here. Predicting document class is thus trivial as we just need to determine which class is more probable than the other. However, one can hardly imagine how difficult the prediction problem is in the real world, especially when predicting tree or graph-like structures and other non-linear structures<sup>3</sup>. For many NLP problems, prediction needs effective and efficient search algorithms.

These problems are general and cover many machine learning and natural language processing tasks. Binary classification, although one of the simplest cases, is sufficient to illustrate the fundamental concepts of machine learning. Classification also has several variants, such as:

- **Multi-class classification.** It is a generalization of binary classification. In multi-class classification, one needs to classify samples into one of three or more classes.
- **Multi-label classification.** This might be confusing because multi-class classification and multi-label classification seem to be the same thing. By conventional use of the terms, multi-label classification is referred to as assigning multiple labels to a sample. By contrast, the problem presented in Eq. (1.41) is a **single-label classification** problem.

Classification would be more interesting and challenging if we extend it to the case of dealing with hierarchical data. For example, for biological and patent data, some classes can be grouped into a super-class. This makes a hierarchy of the classes and requires a hierarchical classification schema.

---

<sup>3</sup>Predicting trees or other structures is not recognized as a standard sub-problem of classification. It is typically referred to as **structure prediction**. We will show in the later sections that both classification and structure prediction can share a similar machine learning paradigm.

## 1.2.2 Documents as Feature Vectors

The first problem we confront in designing text classification models is how to represent a document. Treating  $x$  as a string is simply not a good solution. One may want a representation by which a human being can understand the text. For example, we can parse each sentence in a document into a syntax tree and use trees as a text representation. This, however, requires efforts for developing additional NLP tools (such as syntactic parsers).

Representation, of course, is a fundamental issue in NLP. We omit complex, state-of-the-art models here to present a simple yet effective approach — **the bag-of-words (BOW) model**. The bag-of-words model is a feature-based model of representing documents. In machine learning, a **feature** is a property of a sample. It can be defined not only as a categorical attribute (e.g., name or gender) but also as a numerical value, such as a real number.

In the bag-of-words model, a feature corresponds to the frequency (or count) of a word. Let  $V$  be a vocabulary. A document can be represented as a  $|V|$ -dimensional feature vector. Each dimension corresponds to a word count feature, representing the number of times the  $i$ -th word in  $V$  appears in the document. More formally, let  $\mathbf{x}$  be a feature vector. The  $i$ -th entry of  $\mathbf{x}$  is defined as:

$$x_i = \text{count}(V_i), \quad (1.42)$$

where  $V_i$  is the  $i$ -th word in  $V$  and  $\text{count}(\cdot)$  is a counting function. Consider, for example, the following lines of text<sup>4</sup>.

As I went to Bonner  
I met a pig  
Without a wig,  
Upon my word and honor.

We then have a vocabulary extracted from the corpus<sup>5</sup>, like this

$$V = \{ \text{"a", "and", "as", "Bonner", "honor", "I", "met", "word", "my", "pig", "to", "upon", "went", "wig", "without", ",", ":", "}" \}.$$

Each line of the text can be seen as a document and represented as a feature vector. See below for the feature vectors generated by using the bag-of-words model.

---

<sup>4</sup>The text is from *Mother Goose rhymes*.

<sup>5</sup>We convert all words to lowercase at the beginning of each line.

|                         | a | and | as | Bonner | honor | I | met | word | my | pig | to | upon | went | wig | without | , | . |
|-------------------------|---|-----|----|--------|-------|---|-----|------|----|-----|----|------|------|-----|---------|---|---|
| As I went to Bonner     | 0 | 0   | 1  | 1      | 0     | 1 | 0   | 0    | 0  | 0   | 1  | 0    | 1    | 0   | 0       | 0 | 0 |
| I met a pig             | 1 | 0   | 0  | 0      | 0     | 1 | 1   | 0    | 0  | 1   | 0  | 0    | 0    | 0   | 0       | 0 | 0 |
| Without a wig,          | 1 | 0   | 0  | 0      | 0     | 0 | 0   | 0    | 0  | 0   | 0  | 0    | 0    | 1   | 1       | 1 | 0 |
| Upon my word and honor. | 0 | 1   | 0  | 0      | 1     | 0 | 0   | 1    | 1  | 0   | 0  | 1    | 0    | 0   | 0       | 0 | 1 |

The bag-of-words model defines a vector space<sup>6</sup>. In this space, the similarity between two vectors is measured using metrics such as the dot product. This facilitates establishing relationships between documents: two documents with more overlapping words are considered more similar. This intuitive concept underpins many text classification systems.

The beauty of the bag-of-words model comes from its simple form in that any word is independent of other words. The independence assumption makes it possible to encode a document with almost infinite word relations as a countable, feasibly sized vector. On the other hand, representing a document as a feature vector of word counts is not the only option. As an improvement, one might take more context into account, and/or design more powerful features. This leads to an active line of research on text representation methods, ranging from heuristics-based methods to representation learning methods. We will see a few of them in the subsequent chapters.

### 1.2.3 Linear Classifiers

The **linear classifier** represents one of the most fundamental models in machine learning. Let  $\mathbf{x} \in \mathbb{R}^n$  denote an input feature vector, and let  $\mathbf{w} \in \mathbb{R}^n$  and  $b \in \mathbb{R}$  denote the weight vector and bias parameter, respectively. A linear scoring function is defined as:

$$\begin{aligned}
 s(\mathbf{x}, \mathbf{w}, b) &= \mathbf{w} \cdot \mathbf{x} + b \\
 &= \sum_{i=1}^n w_i x_i + b.
 \end{aligned} \tag{1.43}$$

To simplify notation, we often employ the **bias trick**, defining an augmented input vector  $\mathbf{x}' = [\mathbf{x}; 1]$  and weight vector  $\mathbf{w}' = [\mathbf{w}; b]$ . This allows Eq. (1.43) to be expressed compactly as a dot product:

$$s(\mathbf{x}', \mathbf{w}') = \mathbf{w}' \cdot \mathbf{x}'. \tag{1.44}$$

Henceforth, we will omit the explicit bias term and refer to the parameters simply as  $\mathbf{w}$ .

In the context of classification, the linear function quantifies the affinity between an input and a class. For binary classification with classes  $\mathcal{C} = \{c_a, c_b\}$ , we associate a weight vector with each class, denoted  $\mathbf{w}_a$  and  $\mathbf{w}_b$ .

To perform prediction, we map the continuous scores to discrete class labels using a decision function (often termed an **activation function** or **threshold function**),  $\psi(\cdot)$ . For

<sup>6</sup>A vector space should be closed under vector addition and scalar multiplication.

binary classification, the decision rule is typically:

$$\begin{aligned}\hat{c} &= \psi(s(\mathbf{x}, \mathbf{w}_a) - s(\mathbf{x}, \mathbf{w}_b)) \\ &= \begin{cases} c_a & s(\mathbf{x}, \mathbf{w}_a) > s(\mathbf{x}, \mathbf{w}_b) \\ c_b & \text{otherwise} \end{cases} \end{aligned} \quad (1.45)$$

By linearity,  $s(\mathbf{x}, \mathbf{w}_a) - s(\mathbf{x}, \mathbf{w}_b) = s(\mathbf{x}, \mathbf{w}_a - \mathbf{w}_b)$ . The term  $f(\mathbf{x}) = s(\mathbf{x}, \mathbf{w}_a - \mathbf{w}_b)$  serves as the **linear discriminant function**.

While discriminant functions directly predict class labels, they do not inherently provide probability estimates. To obtain a probabilistic output (i.e.,  $\Pr(c|\mathbf{x})$ ), we must map the scores to a valid probability distribution using a normalization function  $\psi(\cdot)$ , such as the Softmax function:

$$\left[ \Pr(c_a|\mathbf{x}) \ \Pr(c_b|\mathbf{x}) \right] = \left[ \frac{\exp(s(\mathbf{x}, \mathbf{w}_a))}{\exp(s(\mathbf{x}, \mathbf{w}_a)) + \exp(s(\mathbf{x}, \mathbf{w}_b))} \ \frac{\exp(s(\mathbf{x}, \mathbf{w}_b))}{\exp(s(\mathbf{x}, \mathbf{w}_a)) + \exp(s(\mathbf{x}, \mathbf{w}_b))} \right] \quad (1.46)$$

Under this formulation, predicting the class with the highest probability remains equivalent to predicting the class with the highest score, aligning with Eq. (1.45).

Geometrically, a linear classifier defines a **separating hyperplane** (or **decision boundary**) in the feature space. Figure 1.1 depicts this in two dimensions. The objective of learning is to identify a hyperplane (like Hyperplanes 1 and 2) that correctly segregates the data points by class, avoiding the errors made by Hyperplane 3.

It is worthy of noting that linearity is the basis of many classifiers although most of them do not exist in the same form as Eqs. (1.43 - 1.44). A linear model is ideal for **linearly separable** problems. For non-linearly separable data, linear models are often combined with non-linear components, such as non-linear feature transformations or activation functions in neural networks, to capture more complex decision boundaries.

## 1.2.4 Generative vs Discriminative

Classification approaches are generally categorized into two paradigms: **generative models** and **discriminative models**. While both approaches often result in linear decision boundaries, they differ fundamentally in how they model the underlying data distribution.

### 1. Generative Models

A goal of classification is to learn  $\Pr(c|\mathbf{x})$ . Generative models do not explicitly model this conditional probability. Instead, they model the joint probability  $\Pr(\mathbf{x}, c)$ , and use Bayes' rule to compute  $\Pr(c|\mathbf{x})$ . This is given by the following equation.

$$\begin{aligned}\Pr(c|\mathbf{x}) &= \frac{\Pr(\mathbf{x}, c)}{\Pr(\mathbf{x})} \\ &= \frac{\Pr(c) \Pr(\mathbf{x}|c)}{\Pr(\mathbf{x})}, \end{aligned} \quad (1.47)$$

where  $\Pr(\mathbf{x}, c)$  is rewritten as  $\Pr(\mathbf{x}|c) \Pr(c)$ . For an optimal class  $\hat{c}$ , we choose a class  $c$  by

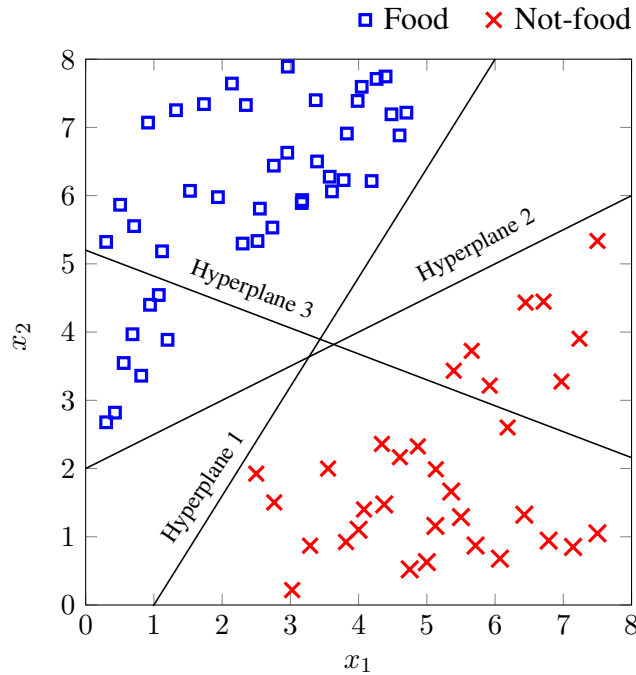


Figure 1.1: Data points and separating hyperplanes in two dimensions. There are two classes of data points (food and non-food). Both hyperplanes 1 and 2 separate the space into two sub-spaces where the two classes of data points are isolated. In this sense, the problem here is linearly separable. On the other hand, hyperplane 3 fails to separate the two classes, that is, the data points in the same class are classified into two different classes.

maximizing  $\Pr(c|\mathbf{x})$  (see Eq. (1.41)). Since the denominator  $\Pr(\mathbf{x})$  is a constant for the class  $c$ , we just need to maximize the numerator. Then, we rewrite Eq. (1.41) in the form

$$\begin{aligned}
 \hat{c} &= \arg \max_{c \in \mathcal{C}} \Pr(c|\mathbf{x}) \\
 &= \arg \max_{c \in \mathcal{C}} \frac{\Pr(c) \Pr(\mathbf{x}|c)}{\Pr(\mathbf{x})} \\
 &= \arg \max_{c \in \mathcal{C}} \Pr(c) \Pr(\mathbf{x}|c),
 \end{aligned} \tag{1.48}$$

where  $\Pr(c)$  is the prior of  $c$ , and  $\Pr(\mathbf{x}|c)$  is the conditional probability of the input document vector  $\mathbf{x}$  given  $c$ . Computing  $\Pr(c)$  is straightforward. For example, the **maximum likelihood estimation (MLE)** defines  $\Pr(c)$  as a relative frequency:

$$\Pr(c) = \frac{\text{count}(c)}{\sum_{c' \in \mathcal{C}} \text{count}(c')}, \tag{1.49}$$

where  $\text{count}(c)$  counts the occurrences of  $c$  in a corpus.

However, estimating  $\Pr(\mathbf{x}|c)$  is challenging because data sparsity prevents accurate esti-

mation for high-dimensional vectors. Recall that the bag-of-words model defines  $x_i$  as the frequency of word  $V_i$  in the document. We assume here that the feature vector  $\mathbf{x} = [x_1, \dots, x_n]$  is generated by a multinomial distribution parameterized by  $(p_1(c), \dots, p_n(c))$ , where  $p_i(c)$  is the probability of word  $V_i$  occurring in class  $c$ . Based on MLE, we estimate  $p_i(c)$  as:

$$p_i(c) = \frac{\text{count}(V_i, c)}{\sum_{1 \leq i' \leq n} \text{count}(V_{i'}, c)}, \quad (1.50)$$

where  $\text{count}(V_i, c)$  is the number of occurrences of  $V_i$  in all documents labeled as  $c$ . Then,  $\Pr(\mathbf{x}|c)$  is given by

$$\Pr(\mathbf{x}|c) = \frac{(\sum_{i=1}^n x_i)!}{x_1! \cdot x_2! \cdot \dots \cdot x_n!} \cdot \prod_{i=1}^n p_i(c)^{x_i}. \quad (1.51)$$

Substituting Eq. (1.51) into Eq. (1.48), we have

$$\hat{c} = \arg \max_{c \in \mathcal{C}} \Pr(c) \cdot \frac{(\sum_{i=1}^n x_i)!}{x_1! \cdot x_2! \cdot \dots \cdot x_n!} \cdot \prod_{i=1}^n p_i(c)^{x_i}. \quad (1.52)$$

Note that  $\frac{(\sum_{i=1}^n x_i)!}{x_1! \cdot x_2! \cdot \dots \cdot x_n!}$  is constant with respect to the class variable  $c$ . We drop it in  $\arg \max$ , and rewrite the right-hand side of the equation in log scale:

$$\hat{c} = \arg \max_{c \in \mathcal{C}} \log(\Pr(c)) + \sum_{i=1}^n x_i \cdot \log(p_i(c)). \quad (1.53)$$

Clearly, this is a linear model. It defines the feature vector and weight vector as below

$$\mathbf{x} = \begin{bmatrix} 1 & x_1 & \dots & x_n \end{bmatrix} \quad (1.54)$$

$$\mathbf{w} = \begin{bmatrix} \log(\Pr(c)) & \log(p_1(c)) & \dots & \log(p_n(c)) \end{bmatrix}. \quad (1.55)$$

Such a form is sometimes called a **log-linear** model, as the linearity comes from transforming the original problem via a logarithmic transformation.

In general, Eq. (1.53) is called the **multinomial naive Bayes** approach. There are, of course, the naive Bayes variants for other types of feature vectors. For example, for binary-valued feature vectors, one can assume a Bernoulli distribution on each entry of a vector and design a **Bernoulli naive Bayes** classifier; for vectors with continuous features, one can assume a Gaussian distribution over continuous data and design a **Gaussian naive Bayes** classifier.

## 2. Discriminative Models

The model defined by  $\Pr(c|\mathbf{x}) = \frac{\Pr(\mathbf{x}, c)}{\Pr(\mathbf{x})} = \frac{\Pr(c) \Pr(\mathbf{x}|c)}{\Pr(\mathbf{x})}$  (see Eq. (1.47)) is called generative because it assumes a specific process for generating data  $\mathbf{x}$  given label  $c$ . The idea is to use the joint probability  $\Pr(\mathbf{x}, c)$  as an intermediate step to compute  $\Pr(c|\mathbf{x})$ . Alternatively, **discriminative models** do not attempt to model the distribution of  $\mathbf{x}$ . Instead, they estimate

$\Pr(c|\mathbf{x})$  directly.

An example is **logistic regression**. Recall that for binary text classification, a naive Bayes classifier predicts label  $c_a$  for a given document  $\mathbf{x}$  only if the log-odds function is positive:

$$f_a(\mathbf{x}) = \log \frac{\Pr(c_a|\mathbf{x})}{\Pr(c_b|\mathbf{x})}. \quad (1.56)$$

We can assume that this quantity follows a linear model:

$$\log \frac{\Pr(c_a|\mathbf{x})}{\Pr(c_b|\mathbf{x})} = \mathbf{w} \cdot \mathbf{x}. \quad (1.57)$$

Since  $\Pr(c_b|\mathbf{x}) = 1 - \Pr(c_a|\mathbf{x})$ , solving for  $\Pr(c_a|\mathbf{x})$  yields:

$$\Pr(c_a|\mathbf{x}) = \frac{1}{1 + \exp(-\mathbf{w} \cdot \mathbf{x})}. \quad (1.58)$$

This represents a sigmoid function, specifically the logistic function. Under this model, we predict  $c_a$  only if  $\mathbf{w} \cdot \mathbf{x} > 0$ . Eq. (1.58) defines the logistic regression classifier, which functions as a discriminative analog to the naive Bayes classifier.

An advantage of discriminative models is the flexibility they offer in framing classification tasks. While generative models attempt to estimate the data distribution of  $\mathbf{x}$ , discriminative models focus on establishing a robust boundary between classes. Discriminative models, therefore, care more about which class is prioritized over another given  $\mathbf{x}$ , or in probabilistic terms, which class is more likely to appear, instead of making strong assumptions on individual data points. This enables model learning by minimizing a specific error metric, rather than strictly maximizing the likelihood of the input data points. This approach falls under the framework of **error-driven learning**.

There are many ways to define these errors. Similar to generative models, one can train a discriminative model by optimizing parameters  $\mathbf{w}$  to maximize the conditional likelihood of the training data. Let  $\{(\mathbf{x}^{(1)}, c^{(1)}), \dots, (\mathbf{x}^{(K)}, c^{(K)})\}$  be a set of labeled documents. The optimal parameter vector  $\hat{\mathbf{w}}$  is given by:

$$\hat{\mathbf{w}} = \arg \max_{\mathbf{w}} \sum_{k=1}^K \log \Pr(c^{(k)}|\mathbf{x}^{(k)}). \quad (1.59)$$

Applying this to Eq. (1.58), the process corresponds to maximizing the log-likelihood:

$$\begin{aligned} \hat{\mathbf{w}} &= \arg \max_{\mathbf{w}} \sum_{k=1}^K \left[ \delta(c_a, c^{(k)}) \log \Pr(c_a|\mathbf{x}^{(k)}) + \delta(c_b, c^{(k)}) \log \Pr(c_b|\mathbf{x}^{(k)}) \right] \\ &= \arg \max_{\mathbf{w}} \sum_{k=1}^K \left[ \delta(c_a, c^{(k)}) \log \left( \frac{1}{1 + \exp(-\mathbf{w} \cdot \mathbf{x}^{(k)})} \right) + \right. \\ &\quad \left. \delta(c_b, c^{(k)}) \log \left( 1 - \frac{1}{1 + \exp(-\mathbf{w} \cdot \mathbf{x}^{(k)})} \right) \right], \end{aligned} \quad (1.60)$$

where  $\delta(\cdot, \cdot)$  is an **indicator function** that returns 1 if the two arguments are equal, and 0 otherwise.

Alternatively, we can train the model by minimizing the 0-1 loss, that is, we count an error whenever the predicted label does not match the true label:

$$\hat{\mathbf{w}} = \arg \min_{\mathbf{w}} \sum_{k=1}^K (1 - \delta(\hat{c}^{(k)}, c^{(k)})), \quad (1.61)$$

where  $\hat{c}^{(k)}$  is the prediction. While mathematically sound, optimizing 0-1 loss directly is difficult because it is non-differentiable, leading to the use of surrogate losses like the log-likelihood mentioned above.

The choice of training objective is crucial in discriminative models. This topic is broad and will be discussed further in Section 1.3.4. Furthermore, discriminative models do not require features to strictly adhere to a probabilistic generative process.  $\mathbf{x}$  can be any feature vector engineered for the task. For example, let  $g(\mathbf{x})$  be the output score of another system. Following Eq. (1.58), a new binary classification model can be constructed:

$$\Pr(c_a|\mathbf{x}) = \frac{1}{1 + \exp(-\mathbf{w} \cdot g(\mathbf{x}))}. \quad (1.62)$$

To learn  $g(\mathbf{x})$ , one can either pre-train it on some additional data, or train it jointly with  $\mathbf{w}$  (i.e., the parameters of the upper-level model  $\Pr(c_a|\mathbf{x})$ ).

### 1.2.5 OOV Words and Smoothing

The **out-of-vocabulary (OOV)** problem occurs when some of the words of a document are not found in the vocabulary. OOV words are common in NLP because new words are always there no matter how much text we have seen. Figure 1.2 gives two curves to illustrate this problem. As shown in Figure 1.2 (a), new words continuously appear when more data is available. When we fix the data that is used for testing the coverage of a vocabulary, OOV words remain even if we have an extremely large vocabulary (Figure 1.2 (b)).

For practical systems, OOV words are common because the vocabulary is often restricted to a “small” number of entries. A standard method is to keep the top- $n$  most frequent words and discard the rest. In this case, OOV words are treated as *unknown* words. For example, a new symbol `<unk>` is introduced into the vocabulary so that all OOV words are denoted as `<unk>`. A more aggressive idea is to build an **open-vocabulary** system that accepts every possible word, but it would require more sophisticated algorithms and probably a task-specific design of data structures. The use of the `<unk>` is still the de facto standard for the development of current NLP systems.

For words that are already in the vocabulary, there are also **unseen words** that are absent in the parameter estimation phase but appear in a new document. A naive implementation of the model described in the previous section would be tough when dealing with unseen words. For example, Eq. (1.50) will simply give a zero probability if the word  $V_i$  does not occur in any training example labeled with  $c$ . In consequence, for a new example containing

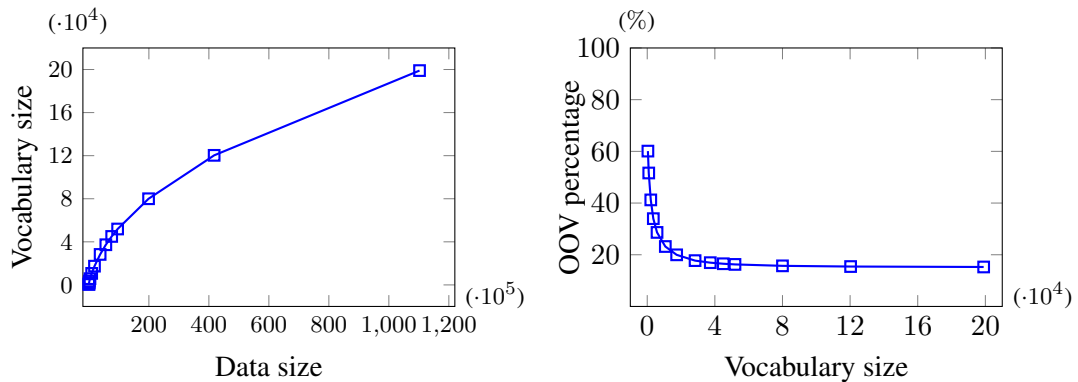


Figure 1.2: Data size (in number of words), vocabulary size and percentage of OOV. The statistics are collected on the English data provided in the WMT21 Zh-En translation task. The more data we use, the larger vocabulary we have. The increase in vocabulary size continues even if we build the vocabulary on data of more than 100 million words. However, the slope of the curve tends to be smaller as more data is involved. Interestingly, the OOV percentage converges to a certain level as the vocabulary size increases, indicating that new words always occur no matter how many words we have observed.

$V_i$ , Eq. (1.51) will assign it a zero probability. This result is obviously unreasonable. However, we should not simply attribute it to the model design itself. Instead, a primary reason for this is the insufficient data used for parameter estimation. This is also explained by **Zipf’s Law**: a small number of words occur quite often, while a large number of words occur rarely.

However, we cannot suppose that we always have access to some data where every word occurs sufficiently. Alternatively, one could adopt smoothing techniques to redistribute the probability over the vocabulary. Consider Eq. (1.50) as an example. We can add a small number to each  $V_i$ , and rewrite the equation as:

$$p_i(c) = \frac{\text{count}(V_i, c) + \alpha}{\sum_{1 \leq i' \leq n} \text{count}(V_{i'}, c) + n \cdot \alpha}, \quad (1.63)$$

where  $\alpha$  is the default count that we assign to each word. In this way,  $p_i(c)$  gives a non-zero probability even if  $\text{count}(V_i, c) = 0$ . Eq. (1.63) shifts probability mass from high-frequency words to low-frequency words. This method is called **additive smoothing** or **add- $\alpha$  smoothing**. It is one of the simplest smoothing methods. For other methods, we refer the reader to language modeling papers where smoothing is in heavy use [Chen and Goodman, 1999].

It is worth noting that in modern NLP, the traditional OOV problem is less frequently discussed. As we will introduce in later chapters, recent NLP systems often use **subword** tokenization methods. In these systems, the processing granularity is no longer traditional linguistic words, but rather statistically learned “units”. Since the vast majority of unseen words can be assembled from these subword units (in the extreme case, words can be treated simply as sequences of characters), using a relatively compact subword vocabulary can significantly alleviate the OOV problem.

## 1.3 General Problems

Building a simple text classifier is a good start but not enough for solving complicated, diverse real-world problems. For a more general picture of how modern machine learning systems work, we now discuss some problems that are important when designing such systems.

### 1.3.1 Supervised and Unsupervised Models

**Supervised learning** deals with labeled samples, by which we mean that an input  $x$  is associated with an output  $y$ . Given a set of input-output pairs  $\{(x^{(1)}, y^{(1)}), \dots, (x^{(K)}, y^{(K)})\}$ , the task here is to learn a function  $f(\cdot)$  that maps each  $x^{(k)}$  to  $y^{(k)}$ :

$$y^{(k)} = f(x^{(k)}). \quad (1.64)$$

This process is called supervised because learning  $f(\cdot)$  is guided by the manually annotated answer  $y^{(k)}$  for  $x^{(k)}$ . In general,  $y^{(k)}$  is referred to as the ground-truth or gold-standard label of  $x^{(k)}$ . After that, when a new input  $x_{\text{new}}$  comes, we use the learned function  $f(\cdot)$  to predict the output. We will say that the supervised learning succeeds if the prediction  $f(x_{\text{new}})$  matches the ground-truth  $y_{\text{new}}$ .

The vast majority of NLP tasks can be framed as supervised learning problems. Assigning a class to a document is undoubtedly one of the simplest cases. Other NLP tasks include, but are not limited to, generating a sequence of labels, a piece of text, a syntax tree, or a graph.

In contrast to supervised learning, **unsupervised learning** deals with unlabeled samples, in other words, for each sample, we have an input  $x$  in the absence of the correct output  $y$ . In this case, we need an algorithm that learns from the unlabeled data  $\{x^{(k)}\}$  the mapping function from  $x^{(k)}$  to some output. Since there is no human intervention on the output of the function, algorithms of this kind need to discover patterns in  $\{x^{(k)}\}$  and optimize the way we represent  $\{x^{(k)}\}$  and function outputs by some criteria. These criteria are typically inspired by human prior knowledge so that the resulting function could output something that we expect.

A common example in unsupervised learning is **word clustering**. It groups a set of words by assigning similar words into the same cluster<sup>7</sup>. Based on such a criterion, we can bias  $f(\cdot)$  towards outputting the same cluster for similar words. A more difficult case is **unsupervised bilingual dictionary induction**. It learns a word-level mapping between two languages without the need of parallel data. The problem is usually addressed, in part, by making use of the isomorphism of word representation spaces of different languages.

Falling halfway between supervised and unsupervised learning is **semi-supervised learning**. It deals with scenarios where only a small portion of the input data is labeled, while the vast majority remains unlabeled. Thus, it leverages the benefits of both paradigms. For example, in machine translation, we may have a certain amount of parallel data (labeled) and orders of magnitude more monolingual data (unlabeled). Often, a base system is trained on the parallel data in a supervised fashion. Improvements can then be made by training components of this base system on the large-scale monolingual data. This is implemented either

<sup>7</sup>It might be difficult and ambiguous to determine if two words are similar or not. We leave this issue to Chapter 3.

by combining a translation model learned on bilingual data with a language model learned on target-language monolingual data, or by pre-training parts of the model on monolingual data and fine-tuning the entire model on the bilingual data.

Broadly speaking, all learning algorithms need supervision. This sounds counterintuitive because unsupervised learning seems to not be signaled by any ground-truth data. However, from a general learning perspective, supervision signals should not be restricted to human-provided labels. Even in an unsupervised learning problem, the learning process is supervised by prior knowledge and the hidden distributional patterns within the input data  $\{x^{(k)}\}$ . In this sense, unsupervised learning is not strictly “learning without supervision”.

Taking “supervision” in this broader sense, additional paradigms emerge as instances of machine learning, though they do not fit neatly into traditional supervised or unsupervised categories. An example is **reinforcement learning**. It models how a system makes a sequence of decisions by operating an agent within an environment. The agent receives feedback (or a reward) from the environment when taking an action. The goal of reinforcement learning is to learn a decision policy that maximizes the cumulative reward over the steps the agent takes. Often, the true reward is only available when the agent reaches a terminal state, such as winning or losing a game. As such, reinforcement learning excels at describing problems characterized by **delayed reward** (or sparse reward). This significantly differentiates it from standard supervised learning, which relies on instant, step-by-step supervision signals encoded in advance. This delayed-reward characteristic fits many NLP problems. For example, a text generation system outputs a sequence of words from left to right, but it is often difficult to determine if a single word was properly chosen until the entire text has been generated and evaluated.

Another example is **self-supervised learning**. It addresses unsupervised learning problems in a supervised learning manner. A general idea is to frame the unsupervised learning task as a pretext task that can be used in solving the original problem. In the pretext task, ground truth can be generated from input data. The learning therefore receives supervision from self-made signals instead of manual labels.

Self-supervised learning has been very successful in NLP. **Pre-training** is perhaps the most prominent application driving recent progress. In pre-training, a base model (such as a language model) is trained via self-supervised learning on massive corpora, and then its learned parameters are applied to a downstream system (such as a sentiment analysis model). This offers two major advantages. First, self-supervised models can be scaled to practically limitless amounts of text because they require no manual labeling. Second, the representations learned during pre-training are highly generalizable and can be applied to a wide range of downstream tasks. In Chapter 7, we will explore several examples of applying self-supervised learning to modern NLP models.

### 1.3.2 Inductive Bias

We informally describe (supervised) machine learning as a process of **inductive reasoning** (or **inductive inference**): we use specific observations (such as labeled documents) to construct a generalized model (such as a classifier). For example, if we observe that the word *cooking*

frequently occurs in documents about food, inductive reasoning leads us to conclude that *cooking* is a strong indicator for all food-related documents. This “specific-to-general” method is simply called **induction**. Once we have an induced model, applying it to predict new, unseen observations is a process of **deduction**.

Induction is the most widely-used principle in designing learners of modern machine learning systems<sup>8</sup>. Imagine that there is a hypothesis space (or model space, or learnable function space) consisting of all possible models that we could make. Learning a model is thus the same as selecting a model in the hypothesis space by inducing from the given training samples. However, we cannot simply assume an oracle model that works well on all unseen samples. A reason for this is that searching for the “best” model in a huge hypothesis space is computationally infeasible. There would be an infinite number of dimensions along which we can design models if the hypothesis space is unconstrained. This is fundamentally related to **the curse of dimensionality**<sup>9</sup>. Another reason is that many models in the hypothesis space can fit training samples well, but only some of them can fit unseen samples. There is a risk of selecting a model that perfectly memorizes the training data but fails to generalize to unseen data. This is relevant to **overfitting**, a concept that we will discuss later.

A natural solution is to define priors on the hypothesis space in a way that allows some models to be more preferable than others. A simple example is that we restrict classifiers to linear models (see Section 1.2.3). This is equivalent to imposing a prior that excludes all non-linear models from the hypothesis space.

Such a prior is known as an **inductive bias**. In a nutshell, an inductive bias is a set of assumptions about the learning problem<sup>10</sup>. For instance, one can assume a specific mathematical form for the model (model bias), choose a particular optimization algorithm (algorithmic bias), or make assumptions about how the data was generated (sample bias)<sup>11</sup>.

Inductive biases try to tell in what way we should describe a problem. Better results are generally favorable when the inductive bias aligns well with the true underlying data distribution. This explains why solutions to some problems prefer certain model architectures (or model biases). Of course, more and stronger inductive biases could make it easier to solve a problem. However, inductive biases are not always helpful, especially when they are not close to the reality.

Let us consider a dice-rolling game to illustrate this. Suppose you have a 6-sided die. Before rolling, you guess a number from 1 to 6, aiming to win as many times as possible. Based on common sense, a random guess is your only logical choice because all sides should have an equal chance of appearing ( $1/6$ ). This holds true for a fair die. However, imagine one day you are playing with a weighted die, and winning becomes difficult. You notice the outcomes no

---

<sup>8</sup>Not all machine learning methods strictly follow induction. Other paradigms include deductive reasoning, abductive reasoning, and analogical reasoning (or transduction) [Hurley, 2011].

<sup>9</sup>The curse of dimensionality refers to the problems that generally appear as the dimensionality of the hypothesis space increases. For example, data sparseness is a common problem that arises when processing high-dimensional data, and is thus a kind of the curse of dimensionality.

<sup>10</sup>A more formal definition can be found in standard machine learning textbooks [Mitchell, 1997].

<sup>11</sup>It is worth noting that the term *bias* has different meanings in other contexts (e.g., bias in neural networks or cognitive bias). We will clarify when a different meaning is intended.

longer follow a uniform distribution. Then, you assume the outcomes follow a multinomial distribution (because you had the experience of developing naive Bayes classifiers). Before betting, you observe 100 practice rolls, choose the most frequent side for future games, and start winning again.

In this example, your initial assumption (i.e., all six sides are equally likely) is a very strong inductive bias because your mental model had zero degrees of freedom. It seemed obvious but failed for the weighted die. Your second inductive bias (the multinomial distribution), though mathematically more complex, is actually a *weaker* assumption. It defines a broader family of models, giving you the flexibility to find the specific probabilities that match the weighted die.

Ultimately, all machine learning models rely on inductive biases. Many of these assumptions are implicit; sometimes, we are not even aware we are making them because they feel entirely “logical”. Yet, if an assumption is fundamentally flawed, it is harmful to the entire learning process. Therefore, practical experience remains essential to recognize and avoid these easily overlooked pitfalls.

### 1.3.3 Non-linearity

Most real-world problems are inherently non-linear, making them difficult to solve with simple linear models. Figure 1.3 provides examples of varying classification difficulty. In Figure 1.3 (a), the two classes can be perfectly separated by a straight line (a hyperplane). In this case, the problem is **linearly separable** because the decision boundary can be represented as a linear function. In contrast, in Figure 1.3 (b), we cannot draw a single hyperplane to perfectly separate the classes. Instead, we require a non-linear decision boundary, such as a hypersphere (a circle in 2D). An even more difficult case is shown in Figure 1.3 (c), where the classes are intertwined and the decision boundary is highly complex.

While modeling non-linear systems introduces computational complexities, several established methods exist to incorporate non-linearity into machine learning systems.

- **Feature mapping and kernel methods.** Recall that a linear classifier can be formulated as a function  $f(\mathbf{w} \cdot \mathbf{x})$ , where  $\mathbf{w}$  is the weight vector,  $\mathbf{x}$  is the feature vector, and  $f(\cdot)$  is the activation function that returns one class (say  $c_a$ ) if its argument is greater than zero and the other class (say  $c_b$ ) otherwise. The idea of feature mapping is to map the feature vector  $\mathbf{x}$  into a higher-dimensional space so that the problem is linearly separable in the new space. For example, let  $\phi(\cdot) : \mathbb{R}^2 \rightarrow \mathbb{R}^4$  be a mapping function and  $\mathbf{x} = \begin{bmatrix} x_1 & x_2 \end{bmatrix}$  be a 2-dimensional vector. We assume that

$$\phi(\begin{bmatrix} x_1 & x_2 \end{bmatrix}) = \begin{bmatrix} x_1^2 + x_2^2 & x_1 & x_2 & 1 \end{bmatrix}. \quad (1.65)$$

By choosing  $\mathbf{w} = \begin{bmatrix} 1 & -8 & -8 & 28 \end{bmatrix}$ , we get a new classifier:

$$\begin{aligned} f(\mathbf{w} \cdot \phi(\mathbf{x})) &= f\left(\begin{bmatrix} 1 & -8 & -8 & 28 \end{bmatrix} \cdot \begin{bmatrix} x_1^2 + x_2^2 & x_1 & x_2 & 1 \end{bmatrix}\right) \\ &= f((x_1 - 4)^2 + (x_2 - 4)^2 - 4). \end{aligned} \quad (1.66)$$

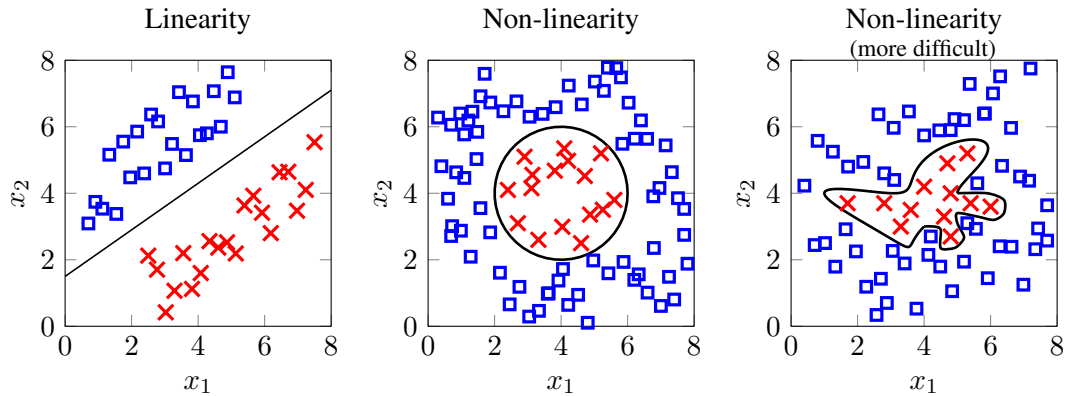


Figure 1.3: Illustrations of linearity and non-linearity in binary classification. The first problem (left) is linearly separable because there exists (at least) a hyperplane that perfectly separates the data points in the two classes. The property of linear separability does not hold in the second problem (middle). Rather, we need a circle-like decision boundary. The decision boundary would be more complex if there are areas where the two classes of the data points are mixed and more or less indistinguishable (right).

It defines a decision boundary (i.e., a hypersphere  $(x_1 - 4)^2 + (x_2 - 4)^2 = 4$ ) that perfectly classifies the samples in Figure 1.3 (b). In other words, we use a linear model on the mapped feature space to create a non-linear model. However, computing the mapping function might be inefficient. This is typically addressed by using kernel methods. In kernel methods, the calculation of vector dot-product in the new space is performed efficiently by using a kernel function in the old space. This method is called **the kernel trick**. It has been successfully adopted in classification and other machine learning models, such as **support vector machines** [Cortes and Vapnik, 1995].

- **Non-linear activation functions.** Another standard approach to introducing non-linearity is through activation functions. A common paradigm is to apply a non-linear activation function to the output of a linear transformation. For instance, the function  $f(\cdot)$  used in the previous example acts as a simple non-linear activation. Depending on the desired output format, various activation functions can be selected. In sophisticated models, multiple non-linear activation functions are embedded within intermediate computational steps to drastically increase the model's expressive power. A **deep neural network**, for example, is essentially a stack of sub-models (commonly referred to as layers), where each layer applies a linear transformation followed by a non-linear activation.
- **Non-parametric methods.** The term *non-parametric* originates from statistics. In non-parametric statistics, statistical inferences are made without any assumption on underlying distributions of data. In machine learning, non-parametric methods follow the same idea. They do not assume any mapping function from input to output as Eq. (1.66). This differentiates them from **parametric methods** that explicitly learn a

mathematical form of variables (or parameters) to describe the problem. An example of non-parametric methods is ***k*-nearest neighbors**, which classifies a new sample based on the majority vote of its  $k$  closest training samples. Note that non-parametric does not mean parameter-free. Rather, it means the number of parameters is not fixed in advance. The complexity of the model grows dynamically as more training data becomes available. As a reward, they can handle highly non-linear problems when training samples are sufficient.

Still, non-linear methods do not work alone. Linearity is an important component of most practical machine learning systems. This has two flavors. First, more non-linearity is not always better. If a linear model sufficiently resolves the problem, introducing non-linear complexity is unnecessary and risks overfitting. In accordance with **Occam's Razor** — *the simplest solution is almost always the best* — most state-of-the-art models maintain a balance of linear and non-linear components. Second, linear models serve as excellent approximations for non-linear behaviors. If the non-linearity of a problem is mild, using linear functions to approximate the solution is often significantly more efficient and interpretable for practical engineering purposes.

### 1.3.4 Training and Loss Functions

Almost all machine learning algorithms involve a training step. Training typically refers to the process of estimating the mapping function and its associated parameters from data. Here, we follow a conventional definition of the training problem: given a model or mapping function, we optimize a specific objective by evaluating the model through some training experience [Mitchell, 1997]. For example, training a naive Bayes text classifier requires maximizing a likelihood function (the objective) over a set of labeled documents (the training experience).

Often, the training problem can be framed as an **optimization** task. As such, we optimize an **objective function** via a training algorithm. Although the ideal objective is the actual performance measure on test samples, we cannot use it directly for optimization, as test samples and their corresponding labels are inaccessible during the training phase. Instead, practical objective functions are defined as computable surrogates for the test-time metric. These objective functions are not necessarily exact performance measures, but rather some metrics that are assumed to correlate with the performance on test data.

Let us consider a general case. Suppose  $\mathbf{y}_\theta = f_\theta(\mathbf{x})$  is a model that takes a feature vector  $\mathbf{x}$  as input and produces an  $n$ -dimensional output vector  $\mathbf{y}_\theta$ . For example, in text classification,  $\mathbf{x}$  is the bag-of-words representation of a document, and  $\mathbf{y}_\theta$  is a probability distribution over a set of classes. Here,  $\theta$  represents the parameters of the model, and the subscript emphasizes that the behavior of the model is determined by  $\theta$ . We further suppose that  $\mathbf{y}_{\text{gold}}$  is the ground-truth (or gold-standard) vector. We then define a **loss function**, denoted as  $\mathcal{L}(\mathbf{y}_\theta, \mathbf{y}_{\text{gold}})$ , which quantifies the discrepancy or error between the prediction  $\mathbf{y}_\theta$  and the true label  $\mathbf{y}_{\text{gold}}$ . It penalizes the model for inaccurate predictions. Given a model architecture, the training problem is described as finding the “best” parameters  $\hat{\theta}$  such that the loss  $\mathcal{L}(\mathbf{y}_\theta, \mathbf{y}_{\text{gold}})$  is

minimized:

$$\hat{\theta} = \arg \min_{\theta} \mathcal{L}(\mathbf{y}_{\theta}, \mathbf{y}_{\text{gold}}). \quad (1.67)$$

This formulation can be easily extended to the case of  $K$  training samples:

$$\hat{\theta} = \arg \min_{\theta} \frac{1}{K} \sum_{k=1}^K \mathcal{L}(\mathbf{y}_{\theta}^{(k)}, \mathbf{y}_{\text{gold}}^{(k)}). \quad (1.68)$$

Once we obtain  $\hat{\theta}$ , we can use  $f_{\hat{\theta}}(\mathbf{x})$  as a fixed model for further predictions.

$\mathcal{L}(\mathbf{y}_{\theta}, \mathbf{y}_{\text{gold}})$  is typically referred to as the **loss function** (computed per sample), while its average over the dataset,  $\frac{1}{K} \sum_{k=1}^K \mathcal{L}(\mathbf{y}_{\theta}^{(k)}, \mathbf{y}_{\text{gold}}^{(k)})$ , is often called the **cost function** or **empirical risk**. A loss function can be defined in many ways, depending on the problem we address and the priors we want to impose upon training. Here, we first consider the case in which  $\mathbf{y}_{\theta}$  is a probability distribution. This is quite common in NLP, for example,  $\mathbf{y}_{\theta}$  could be a distribution over a vocabulary, a list of documents, or a set of syntactic labels. For this type of model output, the most commonly used loss functions are measures of divergence:

- **Divergence-based Loss.** Divergence-based loss functions quantify the difference between the two distributions  $\mathbf{y}_{\theta}$  and  $\mathbf{y}_{\text{gold}}$ . For example, cross-entropy (see Section 1.1.2) is one of the most popular loss functions used in NLP. One can also choose other divergence-based measures, such as the KL divergence and the Jensen-Shannon (JS) divergence, which can be found in most statistics textbooks. Note that the objective of **maximum likelihood estimation (MLE)** is fundamentally tied to divergence-based objectives. Specifically, minimizing the negative log-likelihood (the standard MLE objective) is mathematically equivalent to minimizing the cross-entropy loss when  $\mathbf{y}_{\text{gold}}$  is a one-hot vector (where the entry of the correct label is 1 and all other entries are 0).

However, machine learning systems are not always restricted to distribution-like outputs. Rather,  $\mathbf{y}_{\theta}$  could simply be a vector in  $\mathbb{R}^n$ . An example is the discriminant functions used in classification (see Section 1.2.3). These functions assign a score to each class, indicating how strongly the model believes a specific class is the correct answer. One way to define loss functions for real-valued vectors is to transform them into distribution-like forms<sup>12</sup> and resort to divergence-based losses. However, normalization is not always necessary, especially when we require scores outside the range of  $[0, 1]$ . It is more common to compute losses directly on the raw outputs of these models. Below are some examples.

- **Distance-based Loss.** It is natural to formulate the loss as a geometric distance. A general example is the  $p$ -norm distance (see Section 1.1.1):

$$\mathcal{L}(\mathbf{y}_{\theta}, \mathbf{y}_{\text{gold}}) = \left( \sum_{i=1}^n |y_{\theta}(i) - y_{\text{gold}}(i)|^p \right)^{1/p}. \quad (1.69)$$

---

<sup>12</sup>For example, by normalizing the entries of a vector by the sum of all its entries.

For example, setting  $p = 2$  yields the standard Euclidean distance. Distance-based losses intrinsically describe a **curve fitting** problem: we learn a function  $\mathbf{y}_\theta = f_\theta(\mathbf{x})$  to fit a set of target points  $\{(\mathbf{x}^{(k)}, \mathbf{y}_{\text{gold}}^{(k)})\}$ . This approach is typical in **regression** tasks<sup>13</sup>. A concrete example is the quality estimation of machine translation<sup>14</sup>. The model learns to predict a continuous translation quality score ( $y_\theta$ ) for a source-target sentence pair ( $\mathbf{x}$ ). The prediction is deemed accurate if it closely matches human evaluation. Building upon Eq. (1.69), **mean squared error (MSE)** is a highly popular regression loss. It drops the  $1/p$  root and simply calculates the sum of squared Euclidean distances:

$$\mathcal{L}(\mathbf{y}_\theta, \mathbf{y}_{\text{gold}}) = \sum_{i=1}^n |y_\theta(i) - y_{\text{gold}}(i)|^2. \quad (1.70)$$

Another example is **mean absolute error (MAE)**. It is precisely the form of Eq. (1.69) when  $p = 1$ .

- **0-1 Loss.** The 0-1 loss is widely used in classification problems. It assigns a penalty of 1 for an incorrect prediction and 0 for a correct one. Let  $c_\theta = \arg \max_c y_\theta(c)$  be the predicted label, selected by finding the highest value in  $\mathbf{y}_\theta$ . Likewise, let  $c_{\text{gold}} = \arg \max_c y_{\text{gold}}(c)$  be the ground-truth label. The 0-1 loss is formally defined as:

$$\begin{aligned} \mathcal{L}(\mathbf{y}_\theta, \mathbf{y}_{\text{gold}}) &= \mathcal{L}_{0-1}(c_\theta, c_{\text{gold}}) \\ &= \begin{cases} 1 & c_\theta \neq c_{\text{gold}} \\ 0 & c_\theta = c_{\text{gold}} \end{cases}. \end{aligned} \quad (1.71)$$

- **Margin-based Loss.** A **margin** represents the difference in predicted scores between the correct label  $c_{\text{gold}}$  and an incorrect label  $c$ :

$$\text{margin}(c, c_{\text{gold}}) = y_\theta(c_{\text{gold}}) - y_\theta(c). \quad (1.72)$$

It indicates a distinction between  $c_{\text{gold}}$  and  $c$ . So, a natural idea is to ensure that the margin is sufficiently large, or at least exceeds a minimum. This paradigm is known as **large-margin training**. Let  $\Delta(c, c_{\text{gold}})$  be a predefined penalty cost for misclassifying  $c_{\text{gold}}$  as  $c$ , satisfying  $\Delta(c, c_{\text{gold}}) \geq 0$ , and  $\Delta(c, c_{\text{gold}}) = 0$  if and only if  $c = c_{\text{gold}}$ . Our goal is to maximize the expression  $\text{margin}(c, c_{\text{gold}}) - \Delta(c, c_{\text{gold}})$ ; equivalently, the loss should penalize instances where this value is small or negative. The margin-based loss

<sup>13</sup>When the model output is a vector with two or more dimensions, the problem is known as **multivariate regression**.

<sup>14</sup>In machine translation, quality estimation comprises several sub-tasks. Here, we use the term to refer to predicting an evaluation score directly.

is given by:

$$\begin{aligned}
 \mathcal{L}(\mathbf{y}_\theta, \mathbf{y}_{\text{gold}}) &= \max \left( 0, \max_c -(\text{margin}(c, c_{\text{gold}}) - \Delta(c, c_{\text{gold}})) \right) \\
 &= \max \left( 0, \max_c (y_\theta(c) - y_\theta(c_{\text{gold}}) + \Delta(c, c_{\text{gold}})) \right) \\
 &= \max_c \left( 0, y_\theta(c) - y_\theta(c_{\text{gold}}) + \Delta(c, c_{\text{gold}}) \right). \tag{1.73}
 \end{aligned}$$

Designing  $\Delta(c, c_{\text{gold}})$  depends on the target problem. A simple choice is  $\Delta(c, c_{\text{gold}}) = 1$  for  $c \neq c_{\text{gold}}$ . This formulation makes Eq. (1.73) a variation of the **hinge loss**. An alternative is to use a sum instead of a maximum over all classes  $c$ :

$$\mathcal{L}(\mathbf{y}_\theta, \mathbf{y}_{\text{gold}}) = \sum_c \max \left( 0, y_\theta(c) - y_\theta(c_{\text{gold}}) + \Delta(c, c_{\text{gold}}) \right). \tag{1.74}$$

- **Ranking-based Loss.** Ranking-based loss (or ranking loss) is used in several different areas, such as information retrieval, classification and metric learning. It deals with the problem where we want to order a set of scored items. Suppose the model output  $\mathbf{y}_\theta$  evaluates  $n$  items,  $\{c_i\}$ . We define  $\{\psi_\theta(c_i)\}$  as the predicted ranks of  $\{c_i\}$  sorted by their scores  $\{y_\theta(i)\}$ . For example, given scores  $\{y_\theta(i)\} = \{0.3, -2, 1\}$ , the ranks would be  $\psi_\theta(c_1) = 2$ ,  $\psi_\theta(c_2) = 3$ , and  $\psi_\theta(c_3) = 1$ . We analogously define  $\{\psi_{\text{gold}}(c_i)\}$  as the gold-standard ranks. Note that if a task only requires relative ordering rather than absolute scores,  $\{\psi_{\text{gold}}(c_i)\}$  can be defined without target values  $\mathbf{y}_{\text{gold}}$ . The idea is to penalize ranking inversions in  $\{\psi_\theta(c_i)\}$  compared to  $\{\psi_{\text{gold}}(c_i)\}$ . A straightforward method is to penalize incorrectly ordered item pairs, sharing the idea of binary classification. Let  $\Omega$  be the set of correctly ordered ground-truth pairs:

$$\Omega = \{(i, j) \mid \psi_{\text{gold}}(i) < \psi_{\text{gold}}(j)\}. \tag{1.75}$$

The loss function aggregates penalties over these pairs:

$$\mathcal{L}(\mathbf{y}_\theta, \mathbf{y}_{\text{gold}}) = \sum_{(i, j) \in \Omega} \mathcal{L}_{\text{pair}}(y_\theta(i), y_\theta(j)), \tag{1.76}$$

where  $\mathcal{L}_{\text{pair}}$  is a pairwise classification loss, such as the hinge loss used in [Collobert and Weston, 2008]:

$$\mathcal{L}_{\text{pair}}(y_\theta(i), y_\theta(j)) = \max(0, y_\theta(i) - y_\theta(j) + 1). \tag{1.77}$$

This approach is known as the **pairwise method**. Ranking losses can also be defined in pointwise or listwise manners, which are widely used in modern learning-to-rank systems.

- **Contrastive Loss.** Contrastive loss is typically used in **contrastive learning**. It assumes that, given a sample, there is a similar sample that is labeled as “positive”, and there are a number of dissimilar samples that are labeled as “negative”. A natural idea is

to minimize the distance between similar samples and simultaneously maximize the distance between dissimilar samples. Returning to our formulation, let  $\mathbf{y}^+$  be the positive output and  $\mathbf{Y}^- = \{\mathbf{y}^-\}$  be the set of negative outputs. Also, we use  $\mathbf{y}_{\text{gold}}$  to denote the tuple of  $\mathbf{y}^+$  and  $\mathbf{Y}^-$  instead of a single gold-standard vector. A form of the contrastive loss function is given by the equation:

$$\begin{aligned}\mathcal{L}(\mathbf{y}_\theta, \mathbf{y}_{\text{gold}}) &= \mathcal{L}(\mathbf{y}_\theta, \mathbf{y}^+, \mathbf{Y}^-) \\ &= \log D(\mathbf{y}_\theta, \mathbf{y}^+) - \log \sum_{\mathbf{y}^- \in \mathbf{Y}^-} D(\mathbf{y}_\theta, \mathbf{y}^-) \\ &= \log \frac{D(\mathbf{y}_\theta, \mathbf{y}^+)}{\sum_{\mathbf{y}^- \in \mathbf{Y}^-} D(\mathbf{y}_\theta, \mathbf{y}^-)},\end{aligned}\tag{1.78}$$

where  $D(\alpha, \beta)$  represents a distance metric (e.g., Euclidean distance) between  $\alpha$  and  $\beta$ . A design challenge is generating these positive and negative outputs. In supervised setups, the ground-truth vector naturally serves as the positive target, while negatives can be drawn from other classes. In unsupervised setups,  $\mathbf{y}^+$  and  $\mathbf{Y}^-$  are often defined heuristically. For example, in an **auto-encoding** framework,  $\mathbf{y}^+$  could simply be the input  $\mathbf{x}$  itself (or an augmented view of it), while  $\mathbf{Y}^-$  is a set of randomly generated samples.

- **Error-based Loss.** Evaluation metrics, generally used to quantify system errors, can also be used as loss functions. For example, in machine translation, BLEU is a commonly-used metric (where higher is better, bounded between 0 and 1). One could define the objective as minimizing  $1 - \text{BLEU}$ . Let  $\text{Error}(\mathbf{y}_\theta, \mathbf{y}_{\text{gold}})$  denote the calculated metric error. The error-based loss is simply this value:

$$\mathcal{L}(\mathbf{y}_\theta, \mathbf{y}_{\text{gold}}) = \text{Error}(\mathbf{y}_\theta, \mathbf{y}_{\text{gold}}).\tag{1.79}$$

So far we have presented several loss functions for a wide variety of problems, such as classification, regression, and ranking. As we will see in this book, different loss functions have different effects on model behavior. However, testing all possible loss functions is computationally impractical. Instead, practitioners must carefully select or design the most suitable objective function for their specific task. This process requires time and domain expertise but is essential.

On another hand, there are general methods to improve the design of loss functions. For example, rather than treating the model output  $\mathbf{y}_\theta$  as a deterministic vector, we can model it as a **random variable** drawn from a probability distribution. As a result, the individual loss  $\mathcal{L}(\mathbf{y}_\theta, \mathbf{y}_{\text{gold}})$  also becomes a random variable. Under this framework, we redefine the objective as the **expected loss** (or expected risk) over the distribution of  $\mathbf{y}_\theta$ :

$$\begin{aligned}\mathcal{L}(\mathbf{x}, \mathbf{y}_{\text{gold}}) &= \mathbb{E}_{\mathbf{y}_\theta \sim \text{Pr}(\cdot|\mathbf{x})} [\mathcal{L}(\mathbf{y}_\theta, \mathbf{y}_{\text{gold}})] \\ &= \sum_{\mathbf{y}_\theta} \mathcal{L}(\mathbf{y}_\theta, \mathbf{y}_{\text{gold}}) \cdot \text{Pr}(\mathbf{y}_\theta|\mathbf{x}).\end{aligned}\tag{1.80}$$

By integrating over the entire space of possible predictions  $\mathbf{y}_\theta$ , this method offers a more robust estimation of the loss. Such a paradigm is rooted in **the Bayesian approach**, and  $\mathcal{L}(\cdot)$  (also denoted as  $\mathbb{L}(\cdot)$ ) is commonly referred to as the **Bayesian risk** or simply **expected risk**.

Another way to improve training is introducing priors into the objective. A typical method is to add a regularization term  $R$  to the objective, as follows

$$\hat{\theta} = \arg \min_{\theta} \mathcal{L}(\mathbf{y}_\theta, \mathbf{y}_{\text{gold}}) + \alpha \cdot R \quad (1.81)$$

where  $R(\theta)$  is a function that penalizes certain aspects of the model complexity, such as the magnitude or the number of non-zero parameters. The coefficient  $\alpha$  is a hyperparameter that dictates the strength of the regularization, which balances the trade-off between fitting the training data and maintaining model simplicity. The design of  $R$  is itself a critical area of research in practical machine learning systems. We will explore several specific regularization techniques later in this book.

Once the objective function is defined, an optimization algorithm is required to find the optimal parameters. Optimization is a vast field within machine learning. Hence, a detailed discussion of specific algorithms is beyond the scope of this chapter. Ultimately, there is no single universal algorithm capable of efficiently solving all training problems (a concept related to the “No Free Lunch” theorem). Instead, specific algorithms are chosen based on the mathematical properties of the objective function. For example, one can use *gradient descent* to train a neural language model with a cross-entropy loss [Bengio et al., 2003], use *quadratic programming* to train an SVM with a hinge loss [Cortes and Vapnik, 1995], or use **minimum error-rate training (MERT)** to optimize a statistical machine translation model using a non-differentiable 1 – BLEU loss [Och and Ney, 2002].

### 1.3.5 Overfitting and Underfitting

The standard process of (supervised) machine learning comprises a training step and a test step. While one may try to minimize the loss on training samples, the learned model is used to deal with new samples that are never seen before. This is analogous to a common life experience: a student studying for final exams. While *studying hard = good grades* generally holds true, memorizing textbook questions and answers is a poor strategy for an exam featuring novel questions. Exam questions inevitably differ from the exact examples studied. We therefore need the ability to generalize.

In machine learning, generalization is used to describe how well a model learned through experience predicts on new data. A system is thought to have excellent generalization performance if, guided by appropriate inductive biases, it successfully extracts the underlying patterns from the training data without memorizing the noise. However, achieving good generalization does not mean less training. Instead, practitioners would like to train a machine learning model on more training data to prevent it from memorizing all the things. Generalization is a very complex issue determined by several factors, including problem complexity, model architecture, amount of training data, training algorithm and so on. While there are no standard rules to ensure good generalization, researchers always try to address it somehow.

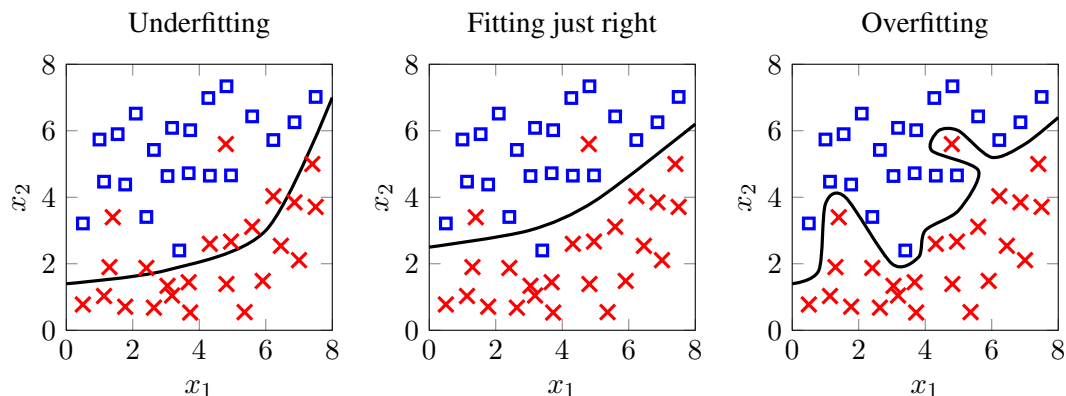


Figure 1.4: Decision boundaries of a binary classification problem. left = underfitting, right = overfitting, and middle = fitting just right. In the underfitting case, there are several obvious mistakes that are made in separating the two classes of data points. By shifting the decision boundary up a bit (middle), we obtain a satisfactory separation result, where most of the data points belonging to the same class are placed on the same side of the decision boundary. By contrast, a perfect separation requires a highly complex decision boundary instead (right).

To describe how well a model generalizes to new data, there are two important concepts, **underfitting** and **overfitting**. Underfitting occurs when a model fails to learn the underlying patterns in the training data sufficiently, resulting in poor performance on both the training and test datasets. This typically happens if the training process is prematurely terminated or if the capacity of the model is simply too low for the task. If a model underfits, one can improve it by increasing the training duration, engineering more expressive features, or using a model with a more complex architecture.

Conversely, overfitting refers to the phenomenon where a model fits the training data exceptionally well but generalizes poorly to test data (see Figure 1.4). In NLP, a basic manifestation of overfitting occurs when a text classifier rigidly memorizes the exact vocabulary seen during training, causing it to fail completely when OOV words appear (see Section 1.2.5).

The causes of overfitting are diverse. A primary cause is training an overly complex model on a small dataset. The model complexity often matters when we design a machine learning model. If the model is complex and has many parameters, then it would be much easier to overfit a small number of samples (see Figure 1.4). The problem would be more difficult if there is noisy data, because of the errors of “garbage in, garbage out” in training. In addition, excessive training is another cause of overfitting. For example, we can heavily tune a system to enforce it to model the data with no errors. The system would be fragile for new samples, even when there are small fluctuations in input.

Overfitting can be alleviated in many ways. Here are some commonly-used techniques.

- **Using more (high-quality) training data.** Large-scale training helps the model capture the true patterns in data. However, adding noisy data would do this in a negative way.
- **Using validation data.** Validation data is a disjoint subset of the available data used to

evaluate the model during the training process, without directly updating the parameters. For example, a dataset can be split into a training set and a **held-out validation set**. Practitioners can monitor performance on the validation set and apply **early stopping**, for example, halting the training process as soon as the validation performance begins to degrade.

- **Using simpler model architectures.** As noted previously, Occam’s Razor is a principle we can follow in model design. If a problem is inherently simple, reducing the model’s capacity (e.g., fewer layers or parameters) forces it to focus on the dominant, generalizable patterns in the data.
- **Regularization.** Regularization is another way to control the model complexity. Typically, it regularizes model parameters by priors. An example is smoothing (see Section 1.2.5). It re-estimates the distribution of words after training. A more general method is regularized training (see Eq. (1.81)). For example, we can define the regularization factor as the  $l_1$  norm of the parameters, and bias the model to those whose parameters are not in large absolute values.
- **Combining multiple models.** A better prediction can also be made by ensembling multiple models. These models (call them **component models**) are in general of different parameters or architectures, and/or are trained with different portions of the data. The variance in models can reduce the risk that all these models overfit the data in exactly the same manner. These models are, therefore, less likely to make similar mistakes in prediction.

### 1.3.6 Prediction and Inference

Although we restricted our discussion to classification in previous sections, (supervised) machine learning is not just a task of predicting a label for an input object. There are many types of machine learning problems, depending on what form of the prediction is defined.

- **Classification.** Classification is perhaps one of the most common machine learning problems. A classification system is required to assign one or more classes to an input object.
- **Regression.** In statistics, regression studies the relationship between a **dependent variable** (or an outcome) and an **independent variable**. While regression has many applications, it is often framed as score prediction in NLP. For example, taking a movie review as input (i.e., an independent variable), the regression model learns to predict a recommendation score (i.e., a dependent variable).
- **Ranking.** A ranking model predicts the order of a set of input objects. For example, a model ranks a number of translations in terms of translation quality.
- **Structure prediction.** Many machine learning models are required to output not only a real value or a class but a tree or a sequence. The task of predicting structured outputs is called structure prediction. For example, a syntactic parser is a structure prediction system, as its output is a tree structure.

Despite being a fundamental aspect of machine learning, prediction is conventionally assumed to be trivial, given that many models and methods are tested on standard classification and regression tasks. On the other hand, prediction is non-trivial in structure prediction tasks, such as parsing and machine translation, which are very common in NLP. Essentially, predicting a tree or a sequence is a **search problem**. For example, there exists a theoretically infinite number of valid translations for a given source-language sentence. Even with a perfect model to evaluate every translation, finding the optimal translation within this search space presents a severe computational challenge.

In such cases, we must employ efficient ways to perform the search. This is implemented by either resorting to general search algorithms from artificial intelligence or developing new heuristics for specific problems. Furthermore, analyzing this search problem offers an important perspective on the mistakes made by a machine learning model: errors can be categorized either as **model errors** (due to inaccurate probability modeling) or as **search errors** (due to the search algorithm failing to find the true optimum). While eliminating search errors is a primary goal, doing so often requires a prohibitively large amount of computational effort. Therefore, deploying a machine learning model for practical purposes invariably requires a trade-off between efficiency and accuracy. We will see examples of this trade-off in Chapter 5.

## 1.4 Model Selection and Evaluation

For most machine learning problems, the goal is to find a model that would perform the best on new data. Two problems can be separated out from this goal [Hastie et al., 2009]:

- **Model selection.** Selecting the best model on training data by some criteria.
- **Model evaluation.** Estimating the performance of a given model on new data.

As noted in Section 1.3.4, loss functions (or error functions) are common ways of measuring errors in a prediction  $\mathbf{y}_\theta = f_\theta(\mathbf{x})$  with respect to a gold-standard  $\mathbf{y}_{\text{gold}}$ . Given  $K$  labeled training samples  $\{(\mathbf{x}^{(1)}, \mathbf{y}_{\text{gold}}^{(1)}), \dots, (\mathbf{x}^{(K)}, \mathbf{y}_{\text{gold}}^{(K)})\}$ , the **training error** is given by

$$\text{Err}_{\text{train}} = \mathcal{L}(\{\mathbf{y}_\theta^{(k)}\}, \{\mathbf{y}_{\text{gold}}^{(k)}\}), \quad (1.82)$$

where  $\{\mathbf{y}_\theta^{(k)}\}$  are the predictions over the training dataset, and  $\{\mathbf{y}_{\text{gold}}^{(k)}\}$  are the corresponding gold-standards.  $\mathcal{L}(\{\mathbf{y}_\theta^{(k)}\}, \{\mathbf{y}_{\text{gold}}^{(k)}\})$  is in general defined as the averaged loss over all training samples:

$$\mathcal{L}(\{\mathbf{y}_\theta^{(k)}\}, \{\mathbf{y}_{\text{gold}}^{(k)}\}) = \frac{1}{K} \sum_{k=1}^K \mathcal{L}(\mathbf{y}_\theta^{(k)}, \mathbf{y}_{\text{gold}}^{(k)}), \quad (1.83)$$

or defined as a single measure on the entire set of training samples. Likewise, we can define the **test error** on the test dataset, denoted as  $\text{Err}_{\text{test}}$ . The test error serves as an estimate of the **generalization error**, indicating how well a model is expected to perform on new data.

In the preceding sections, we assumed that minimizing  $\text{Err}_{\text{train}}$  is the primary objective of

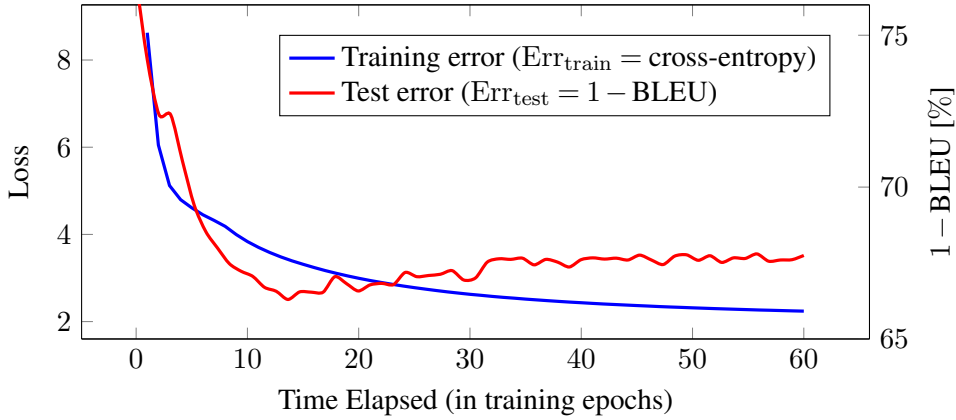


Figure 1.5: Curves of training error and test error for a machine learning system. The training error is measured in terms of the cross-entropy loss, and the test error is measured in terms of  $1 - \text{BLEU}$ . All statistics are collected by running a neural machine translation system on the IWSLT De-En benchmark. The training error continues to drop as more training epochs are involved. The test error, on the other hand, follows a trend of first going down and then going up. When the test error starts to increase, the model is likely to overfit the training data.

training, i.e.,  $\hat{\theta} = \arg \min_{\theta} \text{Err}_{\text{train}}$ . However, we cannot assume that the resulting function  $f_{\hat{\theta}}(\cdot)$  will achieve the minimum  $\text{Err}_{\text{test}}$  simultaneously. Figure 1.5 illustrates the learning curves of a machine translation system. Clearly,  $\text{Err}_{\text{test}}$  does not perfectly correlate with  $\text{Err}_{\text{train}}$ . While the training error decreases continuously as training proceeds, the test error initially follows this downward trend but eventually begins to increase, indicating model overfitting. This makes the problem a bit more complicated, as we cannot always trust  $\text{Err}_{\text{train}}$  although it is and should be the measure of the goodness of training. Surely, we need some way to select a better model, in addition to looking at  $\text{Err}_{\text{train}}$  only.

### 1.4.1 Strategies for Model Selection

Choosing the optimal model based solely on training data is inherently ambitious; we expect a machine learning model to generalize to the entire population from a finite, often small, sample set. From a statistical learning perspective, generalization is only possible because of how we formulate the learning problem. A fundamental assumption in standard machine learning is the i.i.d. assumption — that both the training and test data are independent and identically distributed, generated by the same underlying data-generating distribution.

If this assumption holds and the dataset is sufficiently large, a model that learns from the training data will likely perform well on the test data. By contrast, if the training and test sets are generated in a completely arbitrary, unconstrained manner (e.g., following a uniform distribution over the entire space of all possible functions), the model will fail to generalize. In this case, observing the training data provides absolutely no information about the test data.

This concept is formalized by the **No Free Lunch Theorem** [Wolpert, 1996; Wolpert and Macready, 1997], which proves that if we average performance across all mathemati-

cally possible problems, all learning algorithms perform equally well (equivalent to random guessing)<sup>15</sup>. In other words, universal learning is impossible: an algorithm can only succeed if it incorporates inductive biases that align with the specific problem at hand. Fortunately, in real-world applications, the training and test data is generally assumed to at least in part follow some distribution. Therefore, there are indeed some ways to capture this distribution and improve generalization ability of a model. Two scenarios are generally considered in improving machine learning systems:

- Given a fixed model architecture and training algorithm, how can we develop or curate training data to reduce test error?
- Given a fixed training dataset, how can we design or select a model architecture to reduce test error?

The first scenario is complex and involves practical data engineering issues, such as annotation, data cleaning, and data quality estimation. We will leave these topics for subsequent chapters and focus here on the model selection problem in the second scenario.

## 1. Model Complexity

The simplest method of model selection might be testing the models on validation data. Typically, this data does not overlap with either training or test data, but is assumed to be generated in the same way as the test data. However, such data is not always available. In some cases, we do not even know anything about the test data. So many model selection methods are validation-free.

A common alternative is to rely on **model complexity** (or **model capacity**) as a guiding indicator. In machine learning, model complexity can be interpreted in several different ways. Intuitively, a non-linear model is more complex than a linear one, and a model with millions of parameters is more complex than one with only a few. More formal definitions could be found in computational learning theory, such as the **Vapnik-Chervonenkis (VC) dimension** [Vapnik and Chervonenkis, 1971]. For our purposes, we treat model complexity broadly as a measure of the expressive power of a model: higher complexity equates to a larger hypothesis space.

While highly expressive models are powerful, increased complexity is not strictly beneficial. Complex models are highly susceptible to overfitting, particularly when trained on small datasets. Overly simplistic models, by contrast, are prone to underfitting. The goal of model selection is to identify the “optimal” level of model complexity. Figure 1.6 plots training and test errors against model capacity. Importantly, the optimal complexity is not found where the training error converges to zero, but rather at the point where the validation/test error reaches its global minimum before beginning to rise.

When explicit validation error is unavailable, Occam’s Razor is frequently used: when presented with multiple models that perform comparably well on the training data, the simplest model should be preferred. Many criteria are available to quantify model complexity. For example,

---

<sup>15</sup>The no free lunch theorem was originally presented in a classification scenario [Wolpert, 1996], and was further extended to search and optimization problems [Wolpert and Macready, 1997].

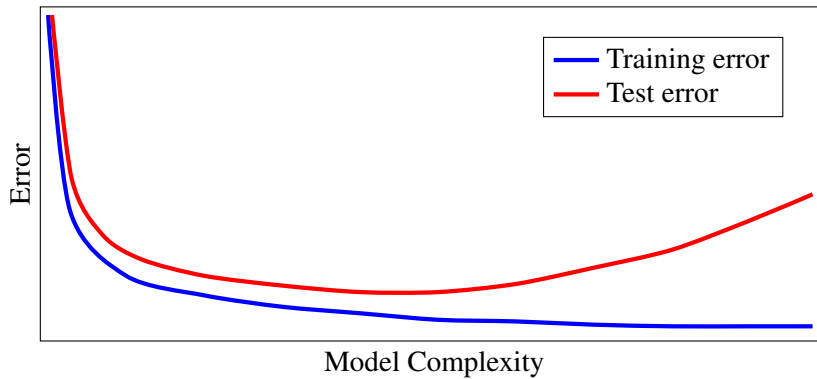


Figure 1.6: Curves of training error and test error under different model complexities. Complex models help in reducing the training error as they can compute complex functions in fitting data points. However, large model complexity is more likely to lead to overfitting and is harmful to the generalization ability of the models. For example, the test error increases as more complexity is added.

- **Number of parameters.** Though very simple, counting the number of free parameters is an intuitive and highly effective heuristic. In linear smoothing models, this is sometimes formalized as the **effective degrees of freedom**, defined as the trace of the projection matrix that maps the ground-truth  $\mathbf{y}_{\text{gold}}$  to the predictions  $\mathbf{y}_{\theta}$ .
- **$p$ -norm of parameters.** The  $p$ -norm of a weight matrix is a strong indicator of complexity (see Section 1.1.1). For example, under an  $L_1$  or  $L_2$  norm constraint, a model with larger absolute parameter values exhibits highly curved decision boundaries and is thus deemed more complex.
- **Description length.** Rooted in information theory and data compression, this evaluates the number of bits required to encode the model and its errors. The **Minimum Description Length (MDL)** principle favors the model that yields the highest degree of data compression.
- **The VC dimension.** It is originally from computational learning theory. The VC dimension formally quantifies capacity as the maximum number of data points that the classifier family can perfectly shatter (assign every possible combination of binary labels to).

In addition, there are other choices for defining the criterion, including **the Akaike information criterion (AIC)**, **the Bayesian information criterion (BIC)**, **the minimum message length (MML)** and so on. They can be found in many textbooks on statistics and/or statistical learning [Burnham and Anderson, 2002; Konishi and Kitagawa, 2007; Hastie et al., 2009].

## 2. Bias-Variance Tradeoff

Controlling model complexity to avoid overfitting and underfitting is also linked to the tradeoff between bias and variance. Bias (or prediction bias) is the amount that the model prediction

differs from the true value. In statistics, bias is a **systematic error** that cannot cancel out even if we run a large number of repeated experiments. In general, bias error results from the wrong assumptions about the problem, such as approximating a non-linear problem via a linear model. This is very interesting! We can establish the connection of the bias error here with the inductive bias used in model design (see Section 1.3.2). For example, given training data, a large bias model is usually due to the fact that there are more assumptions and the model is not complex enough. Simply put, we would say that more (or stronger) inductive biases can result in a lower model complexity and more bias error in prediction. Occasionally, the term *bias* is used as a shorthand for both bias in prediction (from a statistics perspective) and inductive bias (from a model design perspective), although they are considered to have different meanings<sup>16</sup>.

Variance, on the other hand, describes how spread out the predictions are when there are variations in the training data. The variance error also correlates with model complexity. For example, a complex model tends to exhibit higher variance.

Both bias and variance contribute to the overall error of a system. A classic illustration is the bias-variance decomposition of the mean squared error. Here we use some notation that differs slightly from previous sections. Let  $\mathcal{D}$  be a set of  $K$  training samples and  $f_{\hat{\theta}(\mathcal{D})}(\cdot)$  be a model learned on  $\mathcal{D}$ . Furthermore, given a new sample  $\mathbf{x}$ , let  $\mathbf{y}_{\hat{\theta}(\mathcal{D})} = f_{\hat{\theta}(\mathcal{D})}(\mathbf{x})$  be the model prediction and  $\mathbf{y}_{\text{gold}}$  be the “true” prediction. The bias and variance are defined as:

$$\text{bias} = \mathbb{E}_{\mathcal{D}}[\mathbf{y}_{\hat{\theta}(\mathcal{D})}] - \mathbf{y}_{\text{gold}} \quad (1.84)$$

$$\text{variance} = \mathbb{E}_{\mathcal{D}}[\|\mathbb{E}_{\mathcal{D}}[\mathbf{y}_{\hat{\theta}(\mathcal{D})}] - \mathbf{y}_{\hat{\theta}(\mathcal{D})}\|^2], \quad (1.85)$$

where  $\mathbb{E}_{\mathcal{D}}[\mathbf{y}_{\hat{\theta}(\mathcal{D})}]$  is the expected prediction averaged over all possible training datasets of size  $K$ . Thus, the bias represents the deviation between the expected prediction and the true value, while the variance represents the expected squared deviation of the individual predictions from their mean<sup>17</sup>. Taking the mean squared error as the error measure, we can write the expected error as:

$$\begin{aligned} \text{error} &= \mathbb{E}_{\mathcal{D}}[\|\mathbf{y}_{\hat{\theta}(\mathcal{D})} - \mathbf{y}_{\text{gold}}\|^2] \\ &= \|\text{bias}\|^2 + \text{variance}. \end{aligned} \quad (1.86)$$

For lower mean squared error, reducing both bias and variance simultaneously is obviously the ideal goal. However, decreasing one typically increases the other (see Figure 1.7). Practitioners must seek the optimal level of model complexity to balance this tradeoff, thereby preventing overfitting and underfitting. This balance heavily depends on the inherent complexity of the problem. A simple linear model generally exhibits low variance but high bias, which leads to underfitting if the underlying problem is highly non-linear.

<sup>16</sup>Bias is more often used in statistics to describe some aspect of an estimator.

<sup>17</sup>Strictly speaking, bias as defined here is a vector, whereas variance evaluates to a scalar. Furthermore, since  $\mathbf{y}$  represents a probability distribution in this chapter, the discrepancy between distributions is typically measured using divergence metrics (such as cross-entropy or Kullback-Leibler divergence) rather than Euclidean distance. The Euclidean-based bias formulation presented here serves as a mathematically intuitive illustration. Conceptually, one can replace the Euclidean difference with a divergence measure when evaluating distributions.

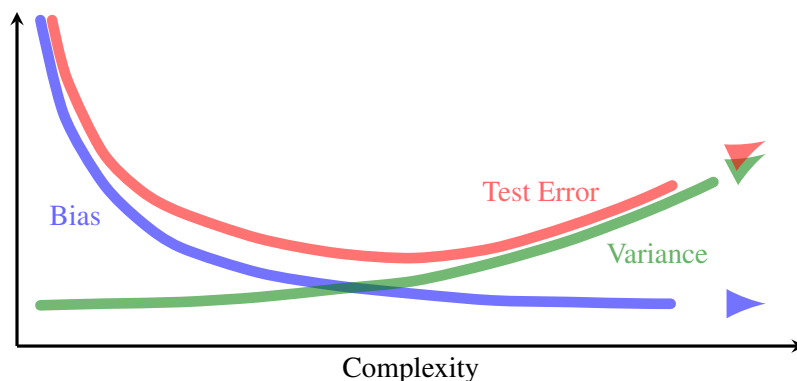


Figure 1.7: Bias and variance against model complexity [Goodfellow et al., 2016]. The curves show a conflict in reducing the bias error and the variance error simultaneously. By varying the model complexity, one can obtain either a low-bias, high-variance model or a high-bias, low-variance model. Both cases exhibit high test error. For example, a high-variance model is often of a larger model complexity. While such a model is able to deal with complex problems, it is more likely to overfit the data. On the other hand, a high-bias model often means a simpler model but tends to underfit the data. To improve generalization on test data, one can seek a tradeoff between bias and variance. For example, there is low test error when a “moderately complex” model is chosen.

Returning to the model selection problem, the bias-variance tradeoff is not a rule for model selection, but a principle guiding model design. Often, one must make compromises to create a model that generalizes well. It is also worth noting that, in many applications, complex models are usually accompanied by computational inefficiency. An appropriate method might be to start with a simple model and only add complexity when it is needed.

### 3. Model Combination

Selecting a single optimal model is not the only strategy to reduce generalization error. Alternatively, one can combine multiple models to form a single, more robust predictor. This paradigm is known as **ensemble learning** [Seni et al., 2010; Zhou, 2012]. The core idea of ensemble learning is to train a set of **component models** (often called base learners) and aggregate their outputs to produce a superior prediction. The simplest approach is **model averaging** (or soft voting), which computes the average of the predicted scores from the individual models. More sophisticated methods, such as mixtures of experts, dynamically weight or combine the internal representations of these sub-models.

In general, component models should exhibit predictive diversity. This diversity is typically achieved by training models on different subsets of the data, using different feature subsets, or applying different weight initializations. From a statistical perspective, these methods can systematically reduce generalization error. For example, bagging helps to lower variance [Breiman, 1996], and boosting helps to lower bias [Schapire, 1990]. These are linked back to what we presented in Section 1.4.1: the generalization error can be reduced by either

reducing the bias error or reducing the variance error.

But discussing how to combine models is beyond the scope of this chapter. Although it may not be strictly appropriate to categorize model combination as a topic related to model selection, it can be seen as a means of improving the generalization ability. In this sense, both model combination and model selection address problems on a similar theme. In fact, model combination is remarkably effective for many NLP tasks. For example, most state-of-the-art systems in NLP are based on the combination of multiple models.

### 1.4.2 Training, Validation and Test Data

We turn now to the data problem. As discussed in the previous sections, in the training stage, a training dataset is used to fit the parameters of the model. In the test stage, a test dataset is used to evaluate the learned model. Closely related to, yet distinct from, the test data is the validation dataset. While a validation dataset is also used for evaluation, it is employed during the training stage. Its primary purpose is to guide model selection and hyperparameter tuning (e.g., determining when to apply early stopping).

In an ideal scenario, researchers have access to separate datasets for training, validation, and testing. However, this is rarely realistic in real-world applications. For example, developers cannot access true future user data before a system is deployed. From a scientific point of view, there is no “real” new data for test — when you see new data, it is not new anymore.

A simple method, as in many research papers, is to verify machine learning models on benchmark tasks. In these tasks, all data is prepared in advance, and all you need is to run your models on the data. Such a method makes it easy to compare different systems directly, as all these systems are trained and tested on the same datasets. Occasionally, we are provided with a single dataset without predefined splits. In such cases, the data can be divided into parts each of which is used for some purposes. For example, a split could be 60% for training, 20% for validation, and 20% for test.

While data splitting provides a way to assess the performance of a model, the assessment result is not always stable due to sampling bias. Sometimes, the performance varies greatly across different runs of data splitting. The problem is more obvious when the dataset is too small to perform sufficient training or test.

A commonly-used way to mitigate the effect of this bias is **cross-validation**. Cross-validation is a resampling method where the data is repeatedly partitioned, and the final result is the average of the assessments over all rounds. A simple variant is random subsampling, which repeatedly randomizes the partition of the data and averages the performance across runs. A more systematic method is  $k$ -fold cross-validation. It divides the data into  $k$  mutually exclusive parts (or folds). In each round of cross-validation,  $k - 1$  parts are combined to form the training data, while the remaining single part is held out as the test (or validation) data. For example, in 10-fold cross-validation, a model is trained and evaluated 10 times, each time using a different fold as the test dataset.

Another note on the scale of data. For practitioners, one of the most frequent questions is how many samples are enough for learning a good model. There is rarely a universal consensus. While computational learning theory provides formal error bounds given a specific sample size,

the prevailing empirical heuristic is simply “the more, the better”. However, modern machine learning increasingly emphasizes **sample efficiency**. A sample-efficient system can achieve a high level of performance using significantly fewer training examples or requiring fewer training epochs. For example, in recent research on NLP, fine-tuning a pre-trained model is highly sample-efficient because it leverages extensive prior knowledge, requiring only modest updates rather than learning representations from scratch. An extreme manifestation of this is **few-shot learning**, which aims to generalize robustly after observing only a handful of task-specific samples.

### 1.4.3 Performance Measure

As an essential part of every machine learning problem, a performance measure describes how well a system performs given some data. Usually it is used in either designing the training objectives or evaluating the result of the final system. For example, all those loss functions described in Section 1.3.4 can serve as performance measures.

When evaluating performance on test data, metrics are often designed to count actual errors. Thus, re-using the loss functions in training might not be a good choice for reporting the final score. For example, the widely-used measures for classification problems are precision, recall and  $F_1$  score. They are proposed to quantify the ability of a classification system in certain aspects: given a class  $c$ , precision computes the fraction of correct predictions among all samples predicted as  $c$ , and recall computes the fraction of correct predictions on all samples labeled as  $c$ . The  $F_1$  score is a measure that combines precision and recall.

Note that performance measures are not necessarily designed for optimization. In this sense, they may not guarantee some mathematical properties, such as being differentiable or continuous. An example is the **BLEU** metric widely used in machine translation [Papineni et al., 2002]. BLEU computes a modified  $n$ -gram precision score combined with a brevity penalty. These discrete counting operations make the metric inherently non-differentiable. NLP features many such task-specific evaluation metrics. This leads to a fundamental training-evaluation discrepancy: the loss function optimized during training differs from the metric used to judge the final model. Practitioners sometimes need to take this gap into account when designing systems.

Another challenge with performance measures in NLP is that a single input often admits multiple valid outputs. For example, there are generally multiple good translations for a source-language sentence. One solution is to take multiple gold-standards into account when designing a performance measure. BLEU is such a case. It counts the maximum number of the correct translation segments over all reference translations. The second solution involves human evaluation. Such a way of evaluation is more accurate but of course is more expensive. When developing practical systems, practitioners usually train and tune the systems using automatic measures, and call for human evaluations for the final test.

### 1.4.4 Significance Tests

Now, assuming you are improving a system in some way, you might be wondering if the improvement is significant enough or not. All you have is a performance measure. So you

can tell the performance difference between any two points in developing the system, but you cannot tell if the difference is real or happens by chance.

In this example, you implicitly try to accept or reject a claim that a system is better than another system (or not). In statistics, **significance tests** provide a formal framework to model this problem. Suppose we evaluate two systems,  $A$  and  $B$ , across multiple datasets or test splits using a consistent performance measure. We formulate two opposing hypotheses:

$H_0$ : System  $A$  performs no better than system  $B$ .

$H_1$ : System  $A$  performs better than system  $B$ .

where  $H_0$  is the **null hypothesis**, and  $H_1$  is the **alternative hypothesis** that is contradictory to the null hypothesis. By testing these hypotheses against empirical data, we aim to determine whether we have sufficient evidence to reject  $H_0$  in favor of  $H_1$ , or if we fail to reject  $H_0$ . Statistical testing is inherently prone to errors: we might incorrectly reject a true null hypothesis (a **Type I error**, or false positive), or we might fail to reject a false null hypothesis (a **Type II error**, or false negative). These two error types trade off against each other. A standard approach in frequentist statistics is to bound the probability of a Type I error below a predefined threshold  $\alpha$ , while seeking to minimize the Type II error. This threshold  $\alpha$  is known as the **significance level** of the test. Common practice in machine learning involves choosing a significance level of 1% or 5%.

When conducting the test, we calculate a  **$p$ -value**. Note that the  $p$ -value is not the probability of making a Type I error. Rather, it is the probability of observing a performance difference as extreme as, or more extreme than, the one actually observed, assuming the null hypothesis is true. If this  $p$ -value is less than the significance level  $\alpha$ , we reject the null hypothesis. For example, with a significance level of 5%, an observed  $p$ -value of 3% indicates that the performance improvement is statistically significant. For a more comprehensive treatment of hypothesis testing, we refer the reader to common statistics textbooks [McClave and Sincich, 2006; Freedman et al., 2007; Freedman, 2009].

Note that the conclusion of significance tests depends on several factors, such as the number of experiments and the variance in the results of experiments. A problem with applying significance tests to NLP tasks is that there are often very few datasets for running the experiments [Dror et al., 2020]. When we can safely assume that the test statistic follows a known probability distribution (such as a Normal distribution), we can employ a parametric test. However, performance of real-world NLP systems rarely adheres to these strict assumptions. As a result, researchers frequently rely on classical **non-parametric tests** (which make no assumptions about the underlying distribution) or empirical resampling methods (such as bootstrapping or approximate randomization). These resampling techniques approximate the underlying distribution by repeatedly sampling the existing dataset or randomly shuffling the system outputs.

Significance tests are important for drawing convincing conclusions in developing machine learning systems, although they are often ignored unintentionally. Figure 1.8 shows evaluation results of three models. Each of them is run for several times with different initial parameters. While system  $A$  is superior to system  $B$  in terms of the averaged performance, there are large

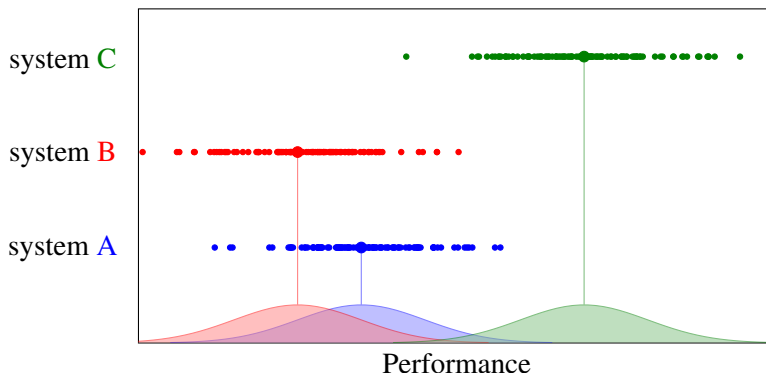


Figure 1.8: Performance of three machine learning systems. For each system, there are many different results because we introduce some randomness into training (e.g., data shuffling, random starting points, etc). Although it seems that System A outperforms System B, there is no real distinction between them, because they overlap a lot in the distributions of the performance (see the bottom of the figure). When comparing System C with System A or B, the difference in performance is significant because we could accept the  $H_1$  hypothesis (i.e., System C outperforms System A or B) given a large number of experiments.

variances in their results. The significance test indicates that the difference is not significant. By contrast, the difference between system A and system C is significant because their performance differs greatly enough in most cases. On the other hand, researchers have found that there are indeed some thresholds of performance gain to indicate significance under certain circumstances. For example, we would say that the significance can be roughly indicated by a certain metric gain if we compare similar systems [Berg-Kirkpatrick et al., 2012].

## 1.5 NLP Tasks as ML Tasks

While there is a wide variety of NLP tasks, many of them can be formulated in a common machine learning framework. This enables a unified paradigm to solve diverse NLP problems using general machine learning approaches. Typically, an NLP task can be described as learning to map language units to some output. Following the notation used in this chapter, we use  $\mathbf{x}$  to denote the input feature vector (or matrix) of an NLP task, and use  $f(\mathbf{x})$  to denote the function that is learned to process  $\mathbf{x}$ . Here are some of the common tasks in NLP.

### 1.5.1 Classification

Suppose there is a set of classes or labels  $\mathcal{C}$ . Each class is represented by a distinct integer in  $\{1, \dots, |\mathcal{C}|\}$ . A classification model is a function that maps the input  $\mathbf{x}$  to a  $|\mathcal{C}|$ -dimensional vector  $\mathbf{y}$ , i.e.,  $\mathbf{y} = f(\mathbf{x})$ . Each entry of  $\mathbf{y}$  is a score corresponding to class  $i$ , denoted by  $y(i)$ . The task here is to assign  $\mathbf{x}$  to one or more classes having the highest scores. Consider

single-label classification as an example. The prediction is given by the equation

$$\hat{c} = \arg \max_{1 \leq i \leq |\mathcal{C}|} y(i), \quad (1.87)$$

where  $\hat{c}$  is the “best” class assigned to  $\mathbf{x}$ . Sometimes, one needs a probability-like output (see Section 1.2.1). Let  $\psi(\cdot)$  be a function that normalizes a vector into a distribution<sup>18</sup>. We then obtain a probabilistic classifier:

$$\mathbf{y} = \psi(f(\mathbf{x})). \quad (1.90)$$

Classification may be the most common problem in NLP. There are many applications in addition to categorizing documents into predefined classes. Among them are choosing a sense for a word [Yarowsky, 1994], determining the polarity of a sentence [Pang et al., 2002], checking whether two entities should be linked [Krebs et al., 2018], classifying the way of associating a semantic argument with a verb [Gildea and Jurafsky, 2002], and so on. When adapting a classification model to these tasks, all you need is to design the form of  $\mathbf{x}$  and the set of classes.

### 1.5.2 Sequence Labeling

An extension to standard classification is to classify a set of samples simultaneously. **Sequence labeling** is an example of such a problem. In sequence labeling, the input is a sequence of  $n$  tokens, such as a sequence of  $n$  words. A sequence labeling system is required to assign each input token  $\mathbf{x}(i)$  a label  $l(i)$ . Here the boldface in  $\mathbf{x}(i)$  is used to emphasize that the token is represented as a feature vector. For convenience, we write  $\mathbf{x}(i)$  as  $\mathbf{x}_i$  and  $l(i)$  as  $l_i$ . The function  $f(\cdot)$  maps the sequence  $\mathbf{x}_1 \cdots \mathbf{x}_n$  into another sequence  $\mathbf{y}_1 \cdots \mathbf{y}_n$ , where  $\mathbf{y}_i$  is the output vector corresponding to  $\mathbf{x}_i$ . This can be formulated as:

$$\begin{bmatrix} \mathbf{y}_1 & \cdots & \mathbf{y}_n \end{bmatrix} = f\left(\begin{bmatrix} \mathbf{x}_1 & \cdots & \mathbf{x}_n \end{bmatrix}\right). \quad (1.91)$$

For vector  $\mathbf{y}_i$ , each entry  $y_i(c)$  corresponds to the prediction score of a class  $c \in \mathcal{C}$ . Note that  $f(\cdot)$  allows for the use of a larger context. For example, one can condition the prediction  $\mathbf{y}_i$  on the entire input sequence [Lafferty et al., 2001]. The final output of the system can be defined as the “optimal” label sequence induced from  $\mathbf{y}_1 \cdots \mathbf{y}_n$ . A simple method is to choose

<sup>18</sup>A similar idea can be found in Eq. (1.46). Given a vector  $\mathbf{a} = [a(1) \cdots a(n)]$ , the normalization function has the form:

$$\psi(\mathbf{a}) = \left[ \frac{a(1)}{\sum_{i=1}^n a(i)} \quad \cdots \quad \frac{a(n)}{\sum_{i=1}^n a(i)} \right] \quad (1.88)$$

Another way is using the Softmax function:

$$\psi(\mathbf{a}) = \left[ \frac{\exp(a(1))}{\sum_{i=1}^n \exp(a(i))} \quad \cdots \quad \frac{\exp(a(n))}{\sum_{i=1}^n \exp(a(i))} \right] \quad (1.89)$$

|              |                                 |                                      |                                      |                                 |                                 |                                 |                    |        |   |
|--------------|---------------------------------|--------------------------------------|--------------------------------------|---------------------------------|---------------------------------|---------------------------------|--------------------|--------|---|
| Tokens :     | Most                            | are                                  | expected                             | to                              | fall                            | below                           | previous-<br>month | levels | . |
| POS tags :   | JJS                             | VBP                                  | VBN                                  | TO                              | VB                              | IN                              | JJ                 | NNS    | . |
| Chunk tags : | $\frac{\text{B-NP}}{\text{NP}}$ | $\frac{\text{B-VP I-VP}}{\text{VP}}$ | $\frac{\text{I-VP I-VP}}{\text{VP}}$ | $\frac{\text{B-PP}}{\text{PP}}$ | $\frac{\text{B-NP}}{\text{NP}}$ | $\frac{\text{I-NP}}{\text{NP}}$ |                    |        | O |

Figure 1.9: An example of sequence labeling for POS tagging and chunking. The example is from the training data of the CoNLL 2000 shared task. Each token is labeled with a POS tag and a chunk tag. A chunk tag has an initial character chosen from  $\{\text{B}, \text{I}, \text{O}\}$ , where B = beginning of a chunk, I = inside a chunk, and O = outside a chunk. So, a chunk always starts with a “B” tag, optionally followed by “I” tags. For example, the VP (verb phrase) chunk in the example spans over the chunk tag sequence “B-VP I-VP I-VP I-VP”.

the label sequence that maximizes the sum of the scores over all positions

$$\begin{bmatrix} \hat{l}_1 & \cdots & \hat{l}_n \end{bmatrix} = \arg \max_{l_1, \dots, l_n \in \mathcal{C}} \sum_{i=1}^n y_i(l_i). \quad (1.92)$$

A straightforward application of sequence labeling to NLP is to tag each token of the input sequence, such as **part-of-speech tagging** (or **POS tagging**). Furthermore, sequence labeling is able to deal with more complex problems by using labels in a clever way. A well-known example is the use of the “IOB” label format in identifying chunks spanning multiple tokens (call it **chunking**). In this method, “I”, “O” and “B” stand for a token inside a chunk, a token outside a chunk, and the leftmost token of a chunk [Ramshaw and Marcus, 1995]. As such, a chunk always starts with a “B” and ends just before the next “B” or a new “O”. See Figure 1.9 for POS tagging and chunking results on an example sentence. As sequence labeling allows the labeling of both tokens and spans, it has been applied with strong results to many tasks, including POS tagging [Bahl and Mercer, 1976], chunking [Tjong Kim Sang and Buchholz, 2000], **named entity recognition** (NER) [Tjong Kim Sang, 2002], and so on.

### 1.5.3 Language Modeling/Word Prediction

**Statistical language modeling** (or **language modeling** for short) is the task of assigning a probability  $\Pr(w_1, \dots, w_n)$  to a sequence of words  $w_1 \cdots w_n$ . This joint probability is generally decomposed into a product of conditional probabilities, by using the chain rule:

$$\begin{aligned} \Pr(w_1, \dots, w_n) &= \Pr(w_1) \cdot \Pr(w_2|w_1) \cdots \Pr(w_n|w_1, \dots, w_{n-1}) \\ &= \prod_{i=1}^n \Pr(w_i|w_1, \dots, w_{i-1}). \end{aligned} \quad (1.93)$$

Eq. (1.93) describes a procedure that generates a word sequence from left to right (call it **auto-regressive** generation). Estimating  $\Pr(w_i|w_1, \dots, w_{i-1})$  is essentially a missing word

prediction problem: we mask out the last word of a sequence and guide the language model to predict the correct word at that position. See below for a word sequence where the last word is missing.

Pride and prejudice is one of the best known \_\_\_\_

We can reuse the idea in classification to model the probability distribution  $\Pr(\text{ } | \textit{Pride, and}, \dots, \textit{known})$ . Let  $\mathbf{x}_i$  be the vector representation of  $w_i$ . We can define a function that reads  $\mathbf{x}_1 \cdots \mathbf{x}_{i-1}$  and produces a vector  $\mathbf{h}_i$ :

$$\mathbf{h}_i = f(\mathbf{x}_1, \dots, \mathbf{x}_{i-1}), \quad (1.94)$$

where  $\mathbf{h}_i$  represents the intermediate state of the model at position  $i$ . To obtain a valid probability distribution, we normalize  $\mathbf{h}_i$  by some normalization function  $\psi(\cdot)$ . Thus, the distribution at position  $i$  would be

$$\begin{aligned} \mathbf{y}_i &= \psi(\mathbf{h}_i) \\ &= \psi(f(\mathbf{x}_1, \dots, \mathbf{x}_{i-1})). \end{aligned} \quad (1.95)$$

Clearly,  $y_i(w_i)$  is the probability of  $w_i$  given previous words, i.e.,  $y_i(w_i) = \Pr(w_i | w_1, \dots, w_{i-1})$ . Note that Eq. (1.93) only considers the left context when predicting a word. A natural extension to this is to condition the prediction on all available context. Consider, for example, a sentence with a masked word in the middle.

Pride and \_\_\_\_ is one of the best-known novels

In this example, we can predict the masked word by using both the left context (*Pride and*) and the right context (*is one of the best known novels*):

$$\mathbf{y}_i = \psi(f(\mathbf{x}_1, \dots, \mathbf{x}_{i-1}, \mathbf{x}_{i+1}, \dots, \mathbf{x}_n)). \quad (1.96)$$

This is a bidirectional model, and is commonly used in **auto-encoding** methods for learning sequence representation models [Devlin et al., 2019].

Note that while language modeling has historically been viewed as a basic task in NLP, it is now recognized as the absolute cornerstone of modern NLP systems. In particular, with the development of large language models, word prediction has emerged as the primary mechanism through which models acquire deep linguistic and world knowledge. As we will see in Chapters 7–11, the language modeling paradigm is extensively used to solve a wide variety of downstream tasks.

### 1.5.4 Sequence Generation

Sequence generation covers a range of NLP problems, including machine translation, summarization, question answering, dialogue systems, and so on. Usually, it refers to mapping some data to a sequence. Here we focus on the **sequence-to-sequence** problem, in that a

source-side sequence is transformed to a target-side sequence, although sequence generation is not specialized to work with chain structures on the source-side.

For notational convenience, we use boldface variables to denote sequences from now on. For example,  $\mathbf{a}$  is a sequence of size  $n$ . It can be written as either  $[a_1 \cdots a_n]$  or  $a_1 \cdots a_n$ . Let  $\mathbf{s} = s_1 \cdots s_m$  and  $\mathbf{t} = t_1 \cdots t_n$  be the source and target sequences, respectively. The sequence-to-sequence problem can be described as finding a target-side sequence that maximizes  $\Pr(\mathbf{t}|\mathbf{s})$ :

$$\hat{\mathbf{t}} = \arg \max_{\mathbf{t}} \Pr(\mathbf{t}|\mathbf{s}). \quad (1.97)$$

Like language modeling,  $\Pr(\mathbf{t}|\mathbf{s})$  can be formalized in an auto-regressive fashion:

$$\begin{aligned} \Pr(\mathbf{t}|\mathbf{s}) &= \Pr(t_1, \cdots, t_n|\mathbf{s}) \\ &= \Pr(t_1|\mathbf{s}) \cdot \Pr(t_2|\mathbf{s}, t_1) \cdots \Pr(t_n|\mathbf{s}, t_1, \cdots, t_{n-1}) \\ &= \prod_{i=1}^n \Pr(t_i|\mathbf{s}, t_1, \cdots, t_{i-1}). \end{aligned} \quad (1.98)$$

Eq. (1.98) differs from Eq. (1.93) only in the additional condition (i.e.,  $\mathbf{s}$ ) introduced to these probabilities. In this sense, we can use Eqs. (1.94-1.95) to model  $\Pr(t_i|\mathbf{s}, t_1, \cdots, t_{i-1})$ . However, conditioning on  $\mathbf{s}$  makes the problem more complex, as we must model the cross-sequence relationship between  $\mathbf{s}$  and  $t_i$ . A straightforward approach to sequence generation is to formulate  $\Pr(t_i|\mathbf{s}, t_1, \cdots, t_{i-1})$  in the **encoder-decoder** paradigm. There are two steps: an encoder is first used to represent  $\mathbf{s}$  as some intermediate form (e.g., a vector), and a decoder is then used to model both the target-side words and the correlation between the encoder output and the target-side words. Putting these together, the output of the encoder-decoder model can be defined to be:

$$\mathbf{y}_i = \text{Decode}(\text{Encode}(\mathbf{s}), t_1, \cdots, t_{i-1}), \quad (1.99)$$

where  $\text{Encode}(\cdot)$  is the encoder, and  $\text{Decode}(\cdot)$  is the decoder.  $\mathbf{y}_i$  is a probability distribution over the target-side vocabulary at position  $i$ , i.e.,  $y_i(t_i) = \Pr(t_i|\mathbf{s}, t_1, \cdots, t_{i-1})$ . Chapter 5 will provide a detailed description of the encoder-decoder model.

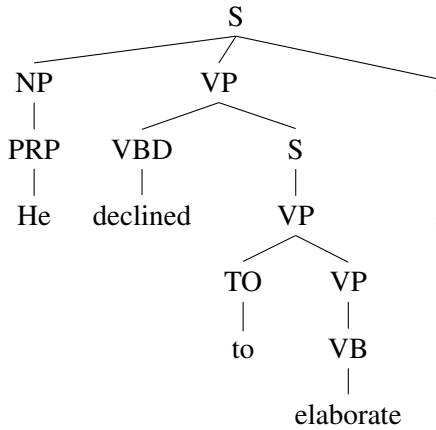
### 1.5.5 Tree Generation

In NLP, trees are usually used to represent the structures or meanings of sequential data. For example, a **syntactic parser** analyzes a sentence to form a **syntax tree** or **parse tree**. More formally, given a sequence of words  $\mathbf{s} = s_1 \cdots s_m$ , the parsing problem can be defined as:

$$\hat{d} = \arg \max_{d \in \mathcal{D}} \Pr(d|\mathbf{s}), \quad (1.100)$$

where  $d$  is a parse tree, and  $\mathcal{D}$  is the set of all parse trees yielding  $s_1 \cdots s_m$ . Computing  $\Pr(d|\mathbf{s})$  is challenging, as the modeling complexity increases exponentially when moving from

Parse Tree:



CFG Rules:

- $r_1 : \text{PRP} \rightarrow \text{He}$
- $r_2 : \text{VBD} \rightarrow \text{declined}$
- $r_3 : \text{TO} \rightarrow \text{to}$
- $r_4 : \text{VB} \rightarrow \text{elaborate}$
- $r_5 : . \rightarrow .$
- $r_6 : \text{NP} \rightarrow \text{PRP}$
- $r_7 : \text{VP} \rightarrow \text{VB}$
- $r_8 : \text{VP} \rightarrow \text{TO VP}$
- $r_9 : \text{S} \rightarrow \text{VP}$
- $r_{10} : \text{VP} \rightarrow \text{VBD S}$
- $r_{11} : \text{S} \rightarrow \text{NP VP } .$

$$\begin{aligned}
 P(d|s) &= P(\text{PRP} \rightarrow \text{He}) \cdot P(\text{VBD} \rightarrow \text{declined}) \cdot P(\text{TO} \rightarrow \text{to}) \cdot P(\text{VB} \rightarrow \text{elaborate}) \cdot \\
 &\quad P(. \rightarrow .) \cdot P(\text{NP} \rightarrow \text{PRP}) \cdot P(\text{VP} \rightarrow \text{VB}) \cdot P(\text{VP} \rightarrow \text{TO VP}) \cdot P(\text{S} \rightarrow \text{VP}) \cdot \\
 &\quad P(\text{VP} \rightarrow \text{VBD S}) \cdot P(\text{S} \rightarrow \text{NP VP } .) \\
 &= \prod_{i=1}^{11} P(r_i)
 \end{aligned}$$

Figure 1.10: An example parse tree and CFG rules. The sentence is from the training data of the CoNLL 2000 shared task. The parse tree is represented as a derivation of CFG rules. The probability of the parse tree is defined as the product of rule probabilities.

sequences to trees. In **statistical parsing**, a solution is to model  $d$  as a derivation of syntactic rules. In this way,  $\Pr(d|s)$  can be formulated as a product of rule probabilities. Figure 1.10 presents an example of parsing with context-free grammar (CFG) rules. Alternatively,  $\Pr(d|s)$  can be modeled in an end-to-end manner. For example, some recent approaches perform parsing by defining a neural network over the parse tree. The probability of a sub-tree rooted at a node is computed by considering the interaction between this node and child nodes.

Another method is to frame parsing as sequence generation. For example, one can linearize a parse tree and represent it as a sequence of words and syntactic labels, or transform the tree generation process into a sequence of actions. This allows the use of sequence-to-sequence techniques in addressing a sequence-to-tree problem.

In linguistics and NLP, tree structures are in heavy use for syntactic analysis. In addition to parsing sentences, they are also applied to represent the internal hierarchical structure of words (morphology), phrases, and entire discourses. However, trees are not the only way to visualize non-linear relationships. A more general data structure is the graph. While trees can be conceptualized as a special class of graphs, there are linguistic phenomena that trees cannot adequately capture [Fellbaum, 2005; Singhal, 2005; Banarescu et al., 2013]. For example, in the semantic representation of a sentence, we often require a graph to connect predicates with their arguments, allowing nodes to possess multiple parents. Although generating general graphs is

computationally more demanding than parsing a sentence into a tree, we can successfully adapt many of the sophisticated methods originally developed for sequence and tree generation.

### 1.5.6 Relevance Modeling

Generally speaking, relevance refers to the degree to which one entity relates to another. The concept of relevance is used in many different sub-fields of NLP and information science. For example, in information retrieval, relevance is used to describe to what extent a retrieved document meets the query. Additional applications of this concept are fundamental to question answering, dialogue systems, semantic analysis, and various other tasks requiring a matching or retrieval mechanism.

Let us consider a more general description. Assume that we have a query and a key that represents something we intend to match with query. Then, we define the feature vectors of query and key as

$$\mathbf{q} = Q(\text{query}) \quad (1.101)$$

$$\mathbf{k} = K(\text{key}), \quad (1.102)$$

where  $Q(\cdot)$  and  $K(\cdot)$  are feature extractors. The relevance between the query and the key is then computed by a scoring function:

$$r = f(\mathbf{q}, \mathbf{k}). \quad (1.103)$$

The function  $f(\cdot)$  can range from a simple distance metric (if  $\mathbf{q}$  and  $\mathbf{k}$  reside in the same vector space) to a complex parameterized model that performs non-linear transformations. In fact, the way of defining relevance can be adopted in several different scenarios. Sometimes, relevance is also termed similarity or correlation. Let  $x$  and  $y$  be two samples (e.g., two words). The similarity of  $x$  and  $y$  is given by

$$r = f(g(x), g(y)), \quad (1.104)$$

where  $g(\cdot)$  is a feature extractor, and  $f(\cdot)$  is a **similarity function**. Learning both  $g(\cdot)$  and  $f(\cdot)$  is called **similarity learning**. In one setup of similarity learning, we fix  $f(\cdot)$  and learn  $g(\cdot)$  in a way that similar samples exhibit similar outputs of  $g(\cdot)$ . Furthermore, the training of the feature extractor does not necessarily need to be coupled with the downstream similarity function. For example, for obtaining the similarity between words, we can pre-train  $g(\cdot)$  via a language modeling objective and subsequently apply it in conjunction with various off-the-shelf similarity functions (such as cosine similarity). This reflects a broader philosophy in modern machine learning: decoupling representation learning from the specific downstream task. Such an idea is widely adopted in pre-training, advancing the recent state-of-the-art on many NLP tasks.

In another setup of similarity learning, we can learn  $f(\cdot)$  directly. This can be performed by either jointly learning  $f(\cdot)$  and  $g(\cdot)$ , or learning  $f(\cdot)$  on top of fixed  $g(\cdot)$ . The problem is also related to **metric learning**. Typically, metric learning is framed as a supervised problem [Kulis,

2013]. A desired similarity function could be learned with the supervision regarding some gold-standard similarity. However, in practice there is usually no such supervised information in NLP. In this case, one could take relative distance as a proxy for supervision. For example, the similarity function can be learned by optimizing a contrastive loss (see Section 1.3.4).

Measuring the similarity between objects plays an important role in many machine learning methods, such as clustering and nearest neighbor classification. In NLP, it is useful for exploring the relationship between words, phrases, sentences, and documents, e.g., similarity is a way to examine how word vectors correspond to our understanding of word meanings [Mikolov et al., 2013; Pennington et al., 2014].

### 1.5.7 Linguistic Alignment

Linguistic alignment is a set of problems where we establish some correspondence between two sets of linguistic units. In NLP, the sequence-to-sequence and sequence-to-tree problems are typically linguistic alignment problems, as they both connect two linguistic units. However, by convention, the term *alignment* refers to aligning multiple objects simultaneously<sup>19</sup>.

As an example, consider the well-known **word alignment** task: we align the words of a sentence to the words of another sentence. We reuse the notation in Section 1.5.4 as both the sequence-to-sequence and word alignment tasks perform on a pair of sequences. Given a source-side word sequence  $\mathbf{s} = s_1 \cdots s_m$  and a target-side word sequence  $\mathbf{t} = t_1 \cdots t_n$ , the word alignment between the two sequences is denoted as an  $m \times n$  matrix  $\mathbf{A}$ .  $A(i, j) = 1$  if there is an **alignment link** between  $s_i$  and  $t_j$ , and  $A(i, j) = 0$  otherwise. The optimal alignment can be defined as:

$$\hat{\mathbf{A}} = \arg \max_{\mathbf{A}} \Pr(\mathbf{A}|\mathbf{s}, \mathbf{t}), \quad (1.105)$$

where  $\Pr(\mathbf{A}|\mathbf{s}, \mathbf{t})$  is the word alignment probability. Like in other machine learning problems, we can model  $\Pr(\mathbf{A}|\mathbf{s}, \mathbf{t})$  in either a generative or discriminative manner (see Section 1.2.4). For example, in Brown et al. [1993]’s work, the word alignment model is factored into several generative steps, each accounting for some assumptions about the problem<sup>20</sup>.

A 0-1 alignment matrix indicates a hard alignment scheme. A problem here is that the hard model may not describe well the highly ambiguous word alignments. We therefore can represent  $\mathbf{A}$  as a real-valued matrix (call it a **soft word alignment matrix** or a **word alignment weight matrix**). Assume that the source-side words are represented as a sequence of feature vectors  $\mathbf{x} = [\mathbf{x}_1 \cdots \mathbf{x}_m]$ . Likewise, the target-side words are represented as  $\mathbf{y} = [\mathbf{y}_1 \cdots \mathbf{y}_n]$ . A soft word alignment model is given by:

$$\mathbf{A} = a(\mathbf{s}, \mathbf{t}), \quad (1.106)$$

<sup>19</sup>The concept of alignment is wide-ranging. We use the term *linguistic alignment* here to differentiate it from the *alignment* of large language models discussed in subsequent chapters.

<sup>20</sup>More precisely, Brown et al. [1993] model  $\Pr(\mathbf{A}, \mathbf{s}|\mathbf{t})$  which is a surrogate of  $\Pr(\mathbf{A}|\mathbf{s}, \mathbf{t})$ , as  $\Pr(\mathbf{A}|\mathbf{s}, \mathbf{t}) = \frac{\Pr(\mathbf{A}, \mathbf{s}|\mathbf{t})}{\Pr(\mathbf{s}|\mathbf{t})} = \frac{\Pr(\mathbf{A}, \mathbf{s}|\mathbf{t})}{\sum_{\mathbf{A}'} \Pr(\mathbf{A}', \mathbf{s}|\mathbf{t})}$

where  $a(\cdot)$  is a word alignment function that computes the alignment weight  $A(i, j)$  for each pair of  $\mathbf{x}_i$  and  $\mathbf{y}_j$ . In fact, all the methods discussed in Section 1.5.6 are applicable to the design of  $a(\cdot)$ . This links the modeling of word alignment with the modeling of similarity, and makes it possible to address different NLP problems by using the same machine learning approach.

Eq. (1.106) offers a very general way to discover the underlying connection over pairs of variables. In addition to aligning words in sequences, it is useful for aligning unordered objects. For example, in bilingual dictionary induction, we can learn such a weight matrix to estimate how strongly a word in one language corresponds to a word in another language.

A final note concerns the role of linguistic alignment models. While linguistic alignment could be thought of as an independent NLP task, it is commonly used in designing sub-models of some downstream systems. Many systems that model word-level relationships involve implicit representation of linguistic alignment. As a consequence, linguistic alignment is treated as some latent states, and is a by-product of these systems. For example, in the early days of statistical machine translation, word alignment was a hidden variable used in modeling the mapping between sequences. The word alignment result can be easily induced from a machine translation model. More recently, neural sequence-to-sequence models — most notably attentional models [Bahdanau et al., 2014] — have attempted to do something similar to word alignment by computing attention weights among words.

### 1.5.8 Information Extraction

In NLP, *extraction* is not a kind of task but a kind of behavior that a system exhibits. Informally, it denotes a process of gathering and distilling structured information from some information sources. So, the term extraction generally appears together with other terms to form a specific task, such as **keyword extraction**, **event extraction**, and **relation extraction**. Many of these tasks can be categorized into an area — **information extraction**. Information extraction is perhaps the broadest topic in NLP. There is no exhaustive list of information extraction tasks. According to Jurafsky and Martin [2008]’s book, it includes but is not limited to named entity recognition, coreference resolution, relation extraction, event extraction, and template filling.

Because information extraction is a heterogeneous collection of diverse problems, it cannot be formulated in a single machine learning paradigm. Fortunately, most of these problems can be framed as standard machine learning problems, such as classification and sequence labeling, and can be solved by using off-the-shelf tools. In some cases, it may require a slight update of existing models for adaptation to a new task. For example, extracting a specific segment from text may require the system to produce a span that indicates the beginning and ending positions of the extracted segment (call it **span prediction**).

On the practical side, machine learning is not always necessary in extracting information from text. Many problems can be solved by using hand-crafted rules. An example is using regular expressions to identify locations and dates in text. In practice real-world systems are usually combinations of heuristic methods and automatic machine learning methods.

### 1.5.9 Remarks

Figure 1.11 shows illustrative examples of the above NLP tasks. Note that we only focused on NLP problems in the supervised learning paradigm. Many unsupervised tasks are important for NLP research as well. For example, it is common to cluster unlabeled words or documents to ease the processing in downstream systems. Several methods are directly applicable to this task [Murphy, 2012]. A recent trend in NLP is that it is not necessary to set a strict boundary between the use of supervised learning and the use of unsupervised learning. In many cases, unsupervised methods help supervised tasks, and vice versa. A notable example is that we learn a pre-trained feature extractor on unlabeled data and build a supervised classifier on top of it. This approach is widely known as **self-supervised learning**: it still follows a supervised learning framework, but the supervision signal is intrinsically derived from the unlabeled data itself. This has led to a paradigm shift toward advancing representation models (i.e., feature extractors) independently, without relying on specific downstream supervised tasks.

## 1.6 Summary

This chapter has given the basic ideas of machine learning and its applications to NLP problems. In particular, we have presented a simple text classification problem to get started with machine learning. Also, we have discussed several general problems on machine learning, e.g., types of machine learning methods, inductive biases, loss functions, overfitting and so on. They are followed by a discussion on model selection and assessment. In addition, we have described how modern NLP problems are framed as machine learning problems.

However, machine learning is a huge research field. Several interesting topics fall outside the scope of this chapter. One such topic that we said little about is reinforcement learning. In general, reinforcement learning is very powerful. It should be and has been considered as an approach to addressing NLP problems, e.g., training a sequence-to-sequence by using a risk-based loss function. A reinforcement learning textbook will offer the general ideas of reinforcement learning [Sutton and Barto, 2018]. Another topic we missed here is Bayesian learning [Gelman et al., 2020; McElreath, 2020; Downey, 2021]. It opens up a notable strand of research in statistical learning, and has been successfully used in NLP tasks. Moreover, there are many other topics that are specialized in certain aspects of machine learning and are of interest to NLP researchers and engineers. Some of them are efficient machine learning [Tay et al., 2020], multi-task learning [Zhang and Yang, 2021], few-shot/zero-shot learning [Wang et al., 2019; 2020], and so on.

A final point to wrap up this chapter. We skip the detailed discussion on certain machine learning models and algorithms, such as classification and regression models, because the reader interested in them can find several excellent, comprehensive introductions [Bishop, 2006; Hastie et al., 2009; Murphy, 2012; Mohri et al., 2018]. In the next chapter we will discuss a bit more about artificial neural networks which are the basis of deep learning and recent state-of-the-art NLP models.

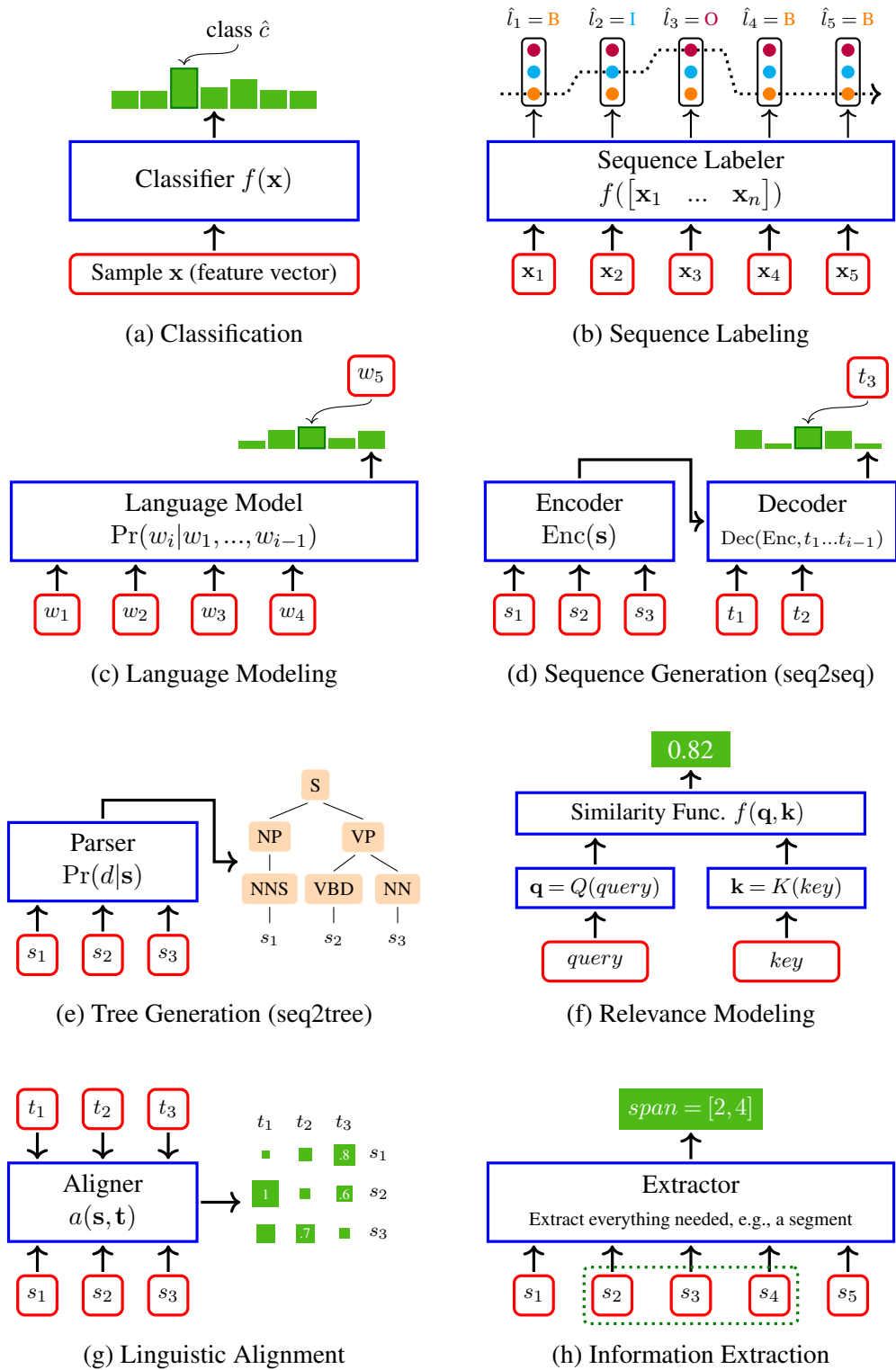


Figure 1.11: Natural language processing tasks from a machine learning perspective.

# Bibliography

- [Bahdanau et al., 2014] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural machine translation by jointly learning to align and translate. *arXiv preprint arXiv:1409.0473*, 2014.
- [Bahl and Mercer, 1976] L. R. Bahl and R. L. Mercer. Part of speech assignment by a statistical decision algorithm. In *Proceedings of IEEE International Symposium on Information Theory*, pages 88–89, 1976.
- [Banarescu et al., 2013] Laura Banarescu, Claire Bonial, Shu Cai, Madalina Georgescu, Kira Griffitt, Ulf Hermjakob, Kevin Knight, Philipp Koehn, Martha Palmer, and Nathan Schneider. Abstract Meaning Representation for sembanking. In *Proceedings of the 7th Linguistic Annotation Workshop and Interoperability with Discourse*, pages 178–186, 2013.
- [Bengio et al., 2003] Yoshua Bengio, Réjean Ducharme, Pascal Vincent, and Christian Jauvin. A neural probabilistic language model. *Journal of Machine Learning Research*, 3:1137–1155, 2003.
- [Berg-Kirkpatrick et al., 2012] Taylor Berg-Kirkpatrick, David Burkett, and Dan Klein. An empirical investigation of statistical significance in NLP. In *Proceedings of the 2012 Joint Conference on Empirical Methods in Natural Language Processing and Computational Natural Language Learning*, pages 995–1005, 2012.
- [Bishop, 2006] Christopher M. Bishop. *Pattern Recognition and Machine Learning*. Springer, 2006.
- [Breiman, 1996] Leo Breiman. Bagging predictors. *Machine Learning*, 24(2):123–140, 1996.
- [Brown et al., 1993] Peter F. Brown, Stephen A. Della Pietra, Vincent J. Della Pietra, and Robert L. Mercer. The mathematics of statistical machine translation: Parameter estimation. *Computational Linguistics*, 19(2):263–311, 1993.
- [Burnham and Anderson, 2002] Kenneth P. Burnham and David R. Anderson. *Model selection and multimodel inference: a practical information-theoretic approach*. Springer, 2002.
- [Chen and Goodman, 1999] Stanley F. Chen and Joshua Goodman. An empirical study of smoothing techniques for language modeling. *Computer Speech and Language*, 13:359–394, 1999.
- [Collobert and Weston, 2008] Ronan Collobert and Jason Weston. A unified architecture for natural language processing: deep neural networks with multitask learning. In *Proceedings of the 25th international conference on Machine learning (ICML08)*, pages 160–167, 2008.
- [Cortes and Vapnik, 1995] Corinna Cortes and Vladimir Vapnik. Support-vector networks. *Mind, Machine Learning*:273–297, 1995.
- [Devlin et al., 2019] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 4171–4186, 2019.

- [Downey, 2021] Allen B. Downey. *Think Bayes: Bayesian Statistics in Python (2nd ed.)*. O'Reilly Media, 2021.
- [Dror et al., 2020] Rotem Dror, Lotem Peled-Cohen, and Segev Shlomov. *Neural Network Methods for Natural Language Processing*. Morgan & Claypool Publishers, 2020.
- [Fellbaum, 2005] Christiane Fellbaum. Wordnet and wordnets. In Keith Brown, editor, *Encyclopedia of Language and Linguistics (2nd ed.)*. Elsevier, 2005.
- [Freedman et al., 2007] David Freedman, Robert Pisani, and Roger Purves. *Statistics (4th ed.)*. W. W. Norton & Company, 2007.
- [Freedman, 2009] David A. Freedman. *Statistical Models: Theory and Practice (2nd ed.)*. Cambridge University Press, 2009.
- [Gelman et al., 2020] Andrew Gelman, John B. Carlin, Hal S. Stern, David B. Dunson, Aki Vehtari, and Donald B. Rubin. *Bayesian Data Analysis (2nd ed.)*. Chapman and Hall/CRC, 2020.
- [Gildea and Jurafsky, 2002] Daniel Gildea and Daniel Jurafsky. Automatic labeling of semantic roles. *Computational Linguistics*, 28(3):245–288, 2002.
- [Goodfellow et al., 2016] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016.
- [Hastie et al., 2009] Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The Elements of Statistical Learning*. Springer, 2009.
- [Hurley, 2011] Patrick Hurley. *A Concise Introduction to Logic (11th ed.)*. Wadsworth Publishing, 2011.
- [Jurafsky and Martin, 2008] Dan Jurafsky and James H. Martin. *Speech and Language Processing (2nd ed.)*. Prentice Hall, 2008.
- [Konishi and Kitagawa, 2007] Sadanori Konishi and Genshiro Kitagawa. *Information Criteria and Statistical Modeling*. Springer, 2007.
- [Krebs et al., 2018] Alicia Krebs, Alessandro Lenci, and Denis Paperno. SemEval-2018 task 10: Capturing discriminative attributes. In *Proceedings of The 12th International Workshop on Semantic Evaluation*, pages 732–740, 2018.
- [Kulis, 2013] Brian Kulis. Metric learning: A survey. *Foundations and Trends® in Machine Learning*, 5(4):287–364, 2013.
- [Lafferty et al., 2001] John Lafferty, Andrew McCallum, and Fernando Pereira. Conditional random fields: Probabilistic models for segmenting and labeling sequence data. In *Proceedings of the 18th International Conference on Machine Learning 2001*, pages 282–289, 2001.
- [McClave and Sincich, 2006] James T. McClave and Terry Sincich. *Statistics (10th ed.)*. Prentice Hall, 2006.
- [McElreath, 2020] Richard McElreath. *Statistical Rethinking: A Bayesian Course with Examples in R and STAN (2nd ed.)*. Chapman and Hall/CRC, 2020.
- [Mikolov et al., 2013] Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg Corrado, and Jeffrey Dean. Distributed representations of words and phrases and their compositionality. In *Proceedings of the 26th International Conference on Neural Information Processing Systems - Volume 2*, pages 3111–3119, 2013.

- [Mitchell, 1997] Tom M. Mitchell. *Machine Learning*. McGraw-Hill Education, 1997.
- [Mohri et al., 2018] Mehryar Mohri, Afshin Rostamizadeh, and Ameet Talwalkar. *Foundations of Machine Learning (2nd ed.)*. MIT Press, 2018.
- [Murphy, 2012] Kevin P. Murphy. *Machine Learning: A Probabilistic Perspective*. MIT Press, 2012.
- [Och and Ney, 2002] Franz Josef Och and Hermann Ney. Discriminative training and maximum entropy models for statistical machine translation. In *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics*, pages 295–302, 2002.
- [Pang et al., 2002] Bo Pang, Lillian Lee, and Shivakumar Vaithyanathan. Thumbs up? sentiment classification using machine learning techniques. In *Proceedings of the 2002 Conference on Empirical Methods in Natural Language Processing (EMNLP 2002)*, pages 79–86, 2002.
- [Papineni et al., 2002] Kishore Papineni, Salim Roukos, Todd Ward, and Wei jing Zhu. Bleu: a method for automatic evaluation of machine translation. In *Proceedings of the 40th Annual meeting of the Association for Computational Linguistics*, pages 311–318, 2002.
- [Pennington et al., 2014] Jeffrey Pennington, Richard Socher, and Christopher D. Manning. Glove: Global vectors for word representation. In *Proceedings of Empirical Methods in Natural Language Processing (EMNLP)*, pages 1532–1543, 2014.
- [Ramshaw and Marcus, 1995] Lance Ramshaw and Mitch Marcus. Text chunking using transformation-based learning. In *Third Workshop on Very Large Corpora*, 1995.
- [Schapire, 1990] Robert E. Schapire. The strength of weak learnability. *Machine Learning*, 5(2): 197–227, 1990.
- [Seni et al., 2010] Giovanni Seni, John Elder, and Robert Grossman. *Ensemble Methods in Data Mining: Improving Accuracy Through Combining Predictions*. Morgan and Claypool Publishers, 2010.
- [Shannon, 1948] Claude E. Shannon. A mathematical theory of communication. Report, Bell Labs, 1948.
- [Singhal, 2005] Amit Singhal. Introducing the knowledge graph: things, not strings, 2005.
- [Sutton and Barto, 2018] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction (2nd ed.)*. The MIT Press, 2018.
- [Tay et al., 2020] Yi Tay, Mostafa Dehghani, Dara Bahri, and Donald Metzler. Efficient transformers: A survey. *CoRR*, abs/2009.06732, 2020.
- [Tjong Kim Sang, 2002] Erik F. Tjong Kim Sang. Introduction to the CoNLL-2002 shared task: Language-independent named entity recognition. In *Proceedings of COLING-02: The 6th Conference on Natural Language Learning 2002 (CoNLL-2002)*, 2002.
- [Tjong Kim Sang and Buchholz, 2000] Erik F. Tjong Kim Sang and Sabine Buchholz. Introduction to the CoNLL-2000 shared task chunking. In *Proceedings of Fourth Conference on Computational Natural Language Learning and the Second Learning Language in Logic Workshop*, 2000.
- [Vapnik and Chervonenkis, 1971] Vladimir Vapnik and Alexey Chervonenkis. On the uniform convergence of relative frequencies of events to their probabilities. *Theory of Probability & Its Applications*, 16(2):264–279, 1971.
- [Wang et al., 2019] Wei Wang, Vincent Wenchen Zheng, Han Yu, and Chunyan Miao. A survey of zero-shot learning. *ACM Transactions on Intelligent Systems and Technology (TIST)*, 10:1–37, 2019.

- [Wang et al., 2020] Yaqing Wang, Quanming Yao, James T. Kwok, and Lionel M. Ni. Generalizing from a few examples: A survey on few-shot learning. *ACM Computing Surveys*, 53(3):1–34, 2020.
- [Wolpert, 1996] David H. Wolpert. The lack of a priori distinctions between learning algorithms. *Neural Computatoin*, 8(7):1341–1390, 1996.
- [Wolpert and Macready, 1997] David H. Wolpert and William G. Macready. No free lunch theorems for optimization. *IEEE Transactions on Evolutionary Computation*, 1(1):67–82, 1997.
- [Yarowsky, 1994] David Yarowsky. Decision lists for lexical ambiguity resolution: Application to accent restoration in Spanish and French. In *Proceedings of the 32nd Annual Meeting of the Association for Computational Linguistics*, pages 88–95, 1994.
- [Zhang and Yang, 2021] Yu Zhang and Qiang Yang. A survey on multi-task learning. *IEEE Transactions on Knowledge and Data Engineering*, pages 1–1, 2021.
- [Zhou, 2012] Zhi-Hua Zhou. *Ensemble Methods: Foundations and Algorithms*. Chapman and Hall/CRC, 2012.